



UNITEC INSTITUTE OF TECHNOLOGY

School of Computing

Master's Thesis

State Estimation of Quadrupeds using Multimodal Sensor Fusion

Sajani Ishara Baladurage

Principal Supervisor: Dr. Jamie Bell

Associate Supervisor: Dr. Russel Mesbah

A thesis submitted in partial fulfilment of the requirements for
the degree of **Master of Applied Technologies**

Date: 8 June 2026

Abstract

The motivation for this project was to enable robot dogs to carry equipment throughout forests in order to assist people who work in the bush. In order for the robots to be able to navigate on rugged terrain, their state estimation algorithms must be robust. State estimation of robots, that move with 6 degrees of freedom, often involves sensor fusion of multiple inputs. These inputs can include accelerometers, gyroscopes, magnetometers, visual odometry, and lidar SLAM. Traditional sensor fusion techniques include various Kalman and particle filter variants. However, such methods do not deal so well with non-linearity and do not detect salient patterns in data that can be used to improve state estimation. As a result, some recent research has applied different types of neural networks for sensor fusion of robots.

The research here performed sensor fusion using neural networks to enhance quadruped odometry. The inputs included accelerometers, gyroscopes, magnetometers, stereo cameras and joint states. The output was the pose change of the quadruped over time. In this research different neural networks were trained and tested in simulation. Also, some of the trained models were tested on a real-world dataset. This research argued for the need and opportunity to experiment with different alternatives for state estimation algorithms, including transformers and 3D CNNs, that measure the odometry of legged robots. The work began with a pilot study comparing two end-to-end visual architectures. A 3D convolutional neural network and a multiscale video transformer were trained across two input modalities: monocular RGB and RAFT-Stereo disparity. The pilot revealed a clear architecture-by-input interaction with the transformer performed better with disparity input while the CNN performed better with RGB, and neither architecture consistently dominated. Scaling to the full dataset further demonstrated that end-to-end transformer training was computationally infeasible, and even the best end-to-end result remained insufficient across all six velocity axes. These findings motivated a transition to a structured preprocessing-and-fusion framework. Three physically grounded preprocessing arms were developed: a forward kinematic estimator producing body-frame velocity from joint states, a Madgwick AHRS filter producing stable orientation features from IMU measurements, and a DROID-SLAM visual odometry pipeline producing $SE(3)$ motion estimates from stereo imagery. A factorized causal transformer was trained to fuse these

representations over time. The proposed fusion system obtained an overall RMSE of 0.074, which is 37% lower than the end-to-end CNN baseline and has 73 times fewer parameters and 88 times less training time. A systematic architecture comparison indicated that the 1D temporal CNN had the highest linear velocity accuracy and the factorized causal transformer had the highest angular velocity accuracy. With the simultaneous presence of visual odometry, a modality ablation showed that the introduction of magnetometer information hurt the performance: the two modalities provide different reference for orientation, which cannot be unambiguously resolved by the model. All experiments were performed using a 1,008 trajectory simulation set gathered in Isaac Sim in forest, flat, and uneven terrain. The simulation dataset, the preprocessing pipeline and the trained models are made available as reusable infrastructure that can be used for future research into legged-robot state estimation. Further testing on the real world data is also left as future work, as the initial testing of the simulation trained models on the real world data produced an overall RMSE of 0.37.

Index Terms:

Quadruped robot, state estimation, multi-sensor fusion, transformer network, 3D convolutional neural network (3D CNN), visual–inertial odometry, rugged terrain, sim-to-real transfer, outdoor robotics

Table of Contents

List of Figures and Tables.....	5
1.0 Introduction.....	7
1.1 Background.....	7
1.2 Aim.....	8
1.3 Research Questions.....	10
1.4 Document Overview.....	11
2.0 Literature Review.....	13
2.1 Filter-Based State Estimation Methods.....	13
2.2 Learning-Based State Estimation Methods.....	15
2.3 Hybrid Sensor Fusion Approaches.....	18
2.4 Visual and Exteroceptive Sensor Integration.....	20
2.5 Multi-Platform and Modular State Estimation Frameworks.....	21
3.0 Research Overview.....	29
3.1 Research Gap.....	29
3.2 Research Questions.....	30
4.0 Methodology.....	32
4.1 Data Collection in Simulation.....	32
4.1.1 Terrain Environments.....	33
4.1.2 Domain Randomization.....	34
4.1.3 Dataset Statistics and Filtering.....	35
4.2 Dataset Split.....	36

4.3 Initial End-to-End Visual Models.....	37
4.3.1 CNN RGB.....	37
4.3.2 MViT RGB	39
4.3.3 Disparity-Based Experiments	40
4.4 Limitations Observed with Initial Approach	41
4.4.1 Training Cost	41
4.4.2 Degraded Performance on Larger Datasets	41
4.4.3 Training Instability.....	42
4.4.4 Poor Scalability of End-To-End Learning	42
4.5 Data Preprocessing Pipeline	42
4.5.1 Joint Kinematics.....	43
4.5.2 Inertial Navigation	44
4.5.3 Visual Odometry	45
4.5.4 Synchronisation, Normalisation, and Clip Construction	46
4.6 Fusion Model Training	47
4.6.1 Training Configurations.....	47
4.7 Fusion Transformer Architecture.....	50
4.8 Intermediate Fusion Models.	50
4.8.1 Fusion_v1.....	50
4.8.2 Fusion_v1_MARG.....	51
4.8.3 FusionTransformers with Visual Odometry – Main Model	52
4.8.4 Fusion TCN Architecture – Alternative Model	54

4.8.5 FusionMViT Architecture- Alternative Model	55
4.9 Testing on Real World Data	57
5.0 Results and Discussion	59
5.1 Initial Pilot Experiments with End-to-End Visual Models	59
5.1.2 Validation Results of Pilot Dataset	59
5.1.3 Test Set Results of Pilot Dataset	60
5.1.4 Architecture Comparison: MViT vs R3D-18 CNN	61
5.1.5 Input Modality Comparison: RGB vs RAFT-Stereo Disparity	62
5.1.6 Validation to Test Generalization	62
5.2 Scaling to the Full Dataset	63
5.2.1 R3D-18 CNN RGB : Baseline Visual Performance	63
5.2.2 Stage-2 R3D-18 CNN RGB : Test Set Evaluation	64
5.2.3 MViT RGB : Validation Performance	65
5.3 Fusion Models.....	67
5.3.1 Fusion v1: Two-Modality Transformer Baseline	67
5.3.2 Fusion v1 MARG: Adding Magnetometer Data.....	68
5.3.3 Fusion v2: Introducing Visual Odometry	70
5.3.4 Fusion v2 MARG: Magnetometer in the Full Three-Modality System.....	72
5.3.5 Fusion TCN: Temporal Convolutional Fusion	74
5.3.6 Fusion MViT: Multiscale Transformer Fusion	75
5.4 Comprehensive Fusion Model Comparison.....	77
5.5 Ablation Studies.....	79

5.5.1 Modality Ablation	80
5.5.2 Temporal Architecture Ablation	81
5.5.3 Single-Modality Baselines	84
5.6 Discussion	86
5.6.1 The Fundamental Limits of Visual Estimation	86
5.6.2 The Value of Structured Preprocessing	87
5.6.3 Signal Redundancy	87
5.6.4 Remaining Error Budget	88
5.7 Real World Test Results	89
5.8 Summary	89
6.0 Conclusion	91
6.1 Question 1: CNN vs Transformers	91
6.2 Question 2: RGB vs RAFT - Stereo Disparity	92
6.3 Question 3: Sim to Real Transfer	92
6.4 Question 4: End to End vs Preprocessing and Fusion.	93
6.5 Question 5 : Optimal Fusion Architecture	94
6.6 Engineering Contribution	94
6.7 Limitation and Future Work	94
6.8 Conclusion	95
Bibliography	97

List of Figures and Tables

Figure 1: Simulated forest like environment	35
Figure 2: Fusion transformer pipeline.....	53
Figure 3: Fusion TCN architecture	55
Figure 4: Fusion MViT diagram	57
Figure 5: The robot and aruco marker board used for data collection.	58
Figure 6: Overall RMSE of all models on test set	78
Figure 7: Compute efficiency	78
Figure 8: Architecture comparison per axis.....	79
Figure 9: Parameter efficiency	79
Figure 10: Modality ablation	80
Figure 11: Single modality vs fusion	85
Table 1: Summary of the Literature Review.....	22
Table 2: Trajectory Counts Per Terrain Type After Stratified Splitting	36
Table 3: Training Configurations.....	48
Table 4: Validation Results of Pilot Dataset.....	60
Table 5: Test Results of Pilot Dataset.....	60
Table 6: Architecture Comparison on Test Dataset	61
Table 7: Input Modality Comparison on Pilot Training	62
Table 8: Validation to Test Generalization	63
Table 9: Stage-2 3D CNN RGB Test Set Performance	64

Table 10:Fusion_v1 Test Set Performance	67
Table 11: Fusion_v1 MARG Test Set Performance	69
Table 12: Fusion_v2 Test Set Performance	70
Table 13:Fusion_v2_marg Test Set Performance	72
Table 14:Fusion_tcn Test Set Performance	74
Table 15:Fusion_mvit Test Set Performance.....	75
Table 16: Fusion Models Comparison	77
Table 17:Summary of Four Relevant Models.....	80
Table 18: Performance Metrics.....	82
Table 19:Single-modality Baselines vs. Fusion_v2.....	84
Table 20:Per-axis Breakdown for Kinematics and IMU	84
Table 21: Real-World Test Results.....	89

1.0 Introduction

1.1 Background

There is a growing demand for quadruped robots in areas where wheeled or tracked platforms cannot traverse. This includes woods, uneven terrains, mountainous areas, and disaster zones [1]. In case of emergency events such as natural disasters, building collapse, or fires, there are limited actions human rescuers can take due to high risk [2], [3]. Quadruped robots can be designed and trained to handle disasters in complex and harsh environments where human access is limited [4], [5]. In New Zealand, the forestry industry and the native bush environment ecosystems have significant importance. They could use quadruped robot assistants in these environments to carry equipment and guide workers in remote and challenging areas [6]. These robots require proper state estimation, or estimating the location, orientation, and movement of the robot, to be effectively and safely operated.

Quadruped robot bodies move with six degrees of freedom (6-DoF), and robust odometry requires integrating information from multiple onboard sensors. The most widely used types of input are inertial measurement units (IMUs: accelerometers and gyroscopes), magnetometers, and visual odometry based on cameras and LiDAR based simultaneous localization and mapping (SLAM) types [7], [8]. Classical sensor fusion techniques, such as the Extended Kalman Filter (EKF) and its variations have been extensively used in legged locomotion [9], [10]. These techniques are both mathematically sound and computationally fast, however they exploit precise models of robot dynamics as well as assumptions of linearity and Gaussian noise. Consequently, they may be noise sensitive, foot slip sensitive, and unmodeled terrain sensitive [11].

Recent studies attempt to eliminate these drawbacks of traditional methods by integrating learning-based methods into the state estimation [12], [13]. It has been explored how neural networks, including recurrent models and convolutional neural networks (CNNs), can synthesize proprioceptive and exteroceptive data, learn non-linear patterns, and implicitly adapt to sensor drift [14], [15]. For example, learning-assisted inertial odometry has demonstrated reduced drift when compared to filter-based methods, and the combination of

neural networks and Kalman filters has demonstrated robustness to challenging terrain and sensor noise [16]. Transformers are also a desirable solution to sequential estimation problems due to their ability to model temporal relationships across a wide distance [17].

Despite the above advancements, there are several challenges that exist. One challenge is determining whether stereo disparity or raw stereo picture pairings offer better learning model features. Another open issue is the sim-to-real gap. Neural networks trained in simulation often fail to generalize to real world deployments because of noise, lighting and dynamic variations [18]. Addressing the above challenges are essential for quadruped robots to achieve reliable state estimation in real outdoor environments such as native bush. The initial goal of this project was to evaluate learning-based state estimation with image-based inputs (raw RGB images vs. stereo disparity). But when scaling to larger datasets, it showed significant performance degradation associated with increased noise and high-dimensional inputs.

In order to overcome this limitation, the focus was moved towards a preprocessing based multimodal fusion approach. With this architecture, each individual sensing modality is first preprocessed through an appropriate means (visual odometry for stereo camera pairs and kinematic models for joint angle sensing) prior to fusing through lightweight neural networks. This approach reduced the training time significantly and improved the robustness in challenging environments.

This project aimed to enhance quadruped odometry by evaluating the performance of preprocessing-based fusion methods, as well as comparing different temporal learning architectures for multimodal sensor integration. Besides advancing the research in the state estimation area, the results contributed to using quadruped robots in forest work and other environmental applications for improving productivity and safety.

1.2 Aim

Developing a strong machine learning-based state estimation framework for quadruped robots that can precisely measure pose changes over time in challenging and unstructured contexts is the main goal of this project. This framework performed multimodal sensor fusion on preprocessed inputs rather than raw sensor data. The inputs for the sensor fusion included

preprocessed features derived from inertial measurement unit, magnetometer, stereo cameras, and joint states.

Early experiments were performed on 12 trajectory sets obtained from the simulated forest environment within Isaac Sim. All four architectures were evaluated during these initial trials. Test RMSEs produced by each architecture ranged between 0.2260 and 0.2572. The MViT model using RAFT-Stereo disparity achieved the lowest test RMSE of 0.2260 among the four pilot configurations. The CNN RGB model produced the second best performance with an RMSE of 0.2328, followed by the CNN RAFT disparity model at 0.2448. The MViT RGB configuration produced the highest test RMSE of 0.2572. This indicated that it was worth pursuing further development across the entire data set.

Scaling to the full data set of 1008 trajectories altered the results. The CNN RGB model trained until early stopping after 10 epochs with a validation RMSE of 0.5173 and an angular RMSE of 0.7138 rad/s. The MViT RGB model completed all fifteen training epochs without significant improvements and ended with a validation RMSE of 0.3977 and an angular RMSE of 0.5574 rad/s. A single run using the CNN architecture took approximately five days on the available GPU. Additionally, MViT was found to be about two-and-a-half times slower per epoch than CNN. It is expected that performing the complete ablation matrix as originally proposed; including RGB vs disparity, CNN vs MViT, and both with or without sensor pretraining, will require many months of continuous use of a GPU and there was no clear way of achieving desired levels of accuracy for real world navigation.

With those limitations in mind, the project switched the focus to a modular feature-based fusion approach. Instead of having one larger network learning all four components (quadruped forward kinematics; egomotion using vision; orientation via IMU; and sensor fusion) simultaneously, each individual sensor based problem is being solved with the classical algorithm that works best for that particular sensor. Then train a lightweight fusion network for the fusion step. Joint States are processed using Forward Kinematics. An IMU is processed using a Madgwick AHRS Filter. Stereo Frames are used to extract SE(3) Pose Estimates using DROID-SLAM. The FusionTransformer takes the Preprocessed Inputs as well as learns how to combine them across both Time and Modalities. In total the Model Size decreased by

approximately 33.5 million parameters to 455k. Training time decreased from 5 Days to approximately 1 hour and 22 minutes. The final fusion_v2 architecture achieved overall Root Mean Squared Error (RMSE) of 0.0737 on the Test Set which consisted of 208 trajectories.

This aim is aligned with the increasing demand to have precise state estimation in legged robots, particularly when used in difficult environments in the real world like forests, where accurate navigation is crucial when performing activities like transporting equipment and supporting human employees. Although the most common variants of the Kalman filter have found extensive use in legged locomotion, they can be difficult to apply in the context of strong nonlinearities, unmodeled terrain interactions, and long-term drift. By contrast, machine learning methods provide opportunities to learn complex patterns in sensor data and improve robustness in conditions where model-based estimators falter.

This paper discusses the main issues in quadruped state estimation, such as successful integration of heterogeneous sensor modalities and the effects of preprocessing on learning-based models. Specifically, it assesses the contribution of the various modalities, such as IMU, kinematics, and visual odometry, to overall performance and whether temporal learning models enhance the accuracy of the estimation.

The output of this research is the trained multimodal fusion models and a thorough analysis of various fusion models, such as multilayer perceptrons, temporal convolutional networks, and transformer-based models. The results indicate that fusion using preprocessing can greatly enhance the accuracy of state estimation at a minimal cost. Such contributions will be used to create robust quadruped robots to be used in difficult outdoor operations like forestry and conservation missions.

1.3 Research Questions

Original research questions:

1. Does a 3D CNN (Video ResNet [19]) or a video transformer (Video MViT) perform quadruped state (6 Degree of Freedom Velocity) estimation more

accurately, when traversing rugged terrain using an IMU, magnetometer, stereo camera and joint states?

2. When using stereo cameras for this state estimation, does inputting RGB images or preprocessed stereo disparity produce more accurate results and what is the difference?
3. Does a state estimation neural network for a quadruped- using an IMU, magnetometer, joint states and stereo cameras as input- trained in simulation (with real world data input and domain randomisation) transfer to equivalent accuracy in the real world? If not, what is the accuracy difference?

Amended research questions:

4. Is state estimation more accurate with an architecture that is trained end-to-end learning or by preprocessing the sensors input before sensor fusion (using neural networks for the IMU and magnetometer, joint odometry for the legs and SLAM for the stereo-cameras)?
5. Which architecture produces the most accurate state estimation when preprocessing sensor inputs?

1.4 Document Overview

This report consists of six main sections.

Section 1: Gives an introduction to the research, the background, the purpose, and research question.

Section 2: Gives a literature review, which recaps the existing solutions to quadruped state estimation and points out their drawbacks. It highlights areas of research gaps, including the absence of a viable solution to unstructured bush environments.

Section 3: Presents the proposed research, identified research gaps and research questions

Section 4: Outlines the methodology used in the experiment. This includes the Isaac Sim simulation setup, the terrain curriculum, domain randomization, data collection and filtering, the data split and the training processes of both the end-to-end visual models and the FusionTransformer pipeline. It also describes the real-world dataset used.

Section 5: gives results and discussion. The original results of the end-to-end visual model are discussed in section 5.1 and 5.2. The results of the fusion models and the comparison of all the models are discussed in Section 5.3 to 5.8.

Section 6: Summarized the contribution of this research, limitations and provides future work directions.

The document ends with the bibliography of all referenced works.

2.0 Literature Review

There are a number of studies that use different approaches for state estimation of robots. This literature review investigates those approaches by analyzing the experiments, achieved results, and future work that needs to be explored.

2.1 Filter-Based State Estimation Methods

Yu et al. [20] performed a study about quadruped robots using 37 sensors, including joint force and displacement sensors, foot end sensors, and an IMU. Simulations are conducted using RecurDyn and Matlab. The paper presents the moving horizon estimation approach, which combines IMU and leg kinematics data to estimate the robot's state and velocity. Experiments were conducted injecting noise into IMU and kinematic data to simulate the real-world disturbances, and the performance of MHE based fusion techniques was improved compared to the performance of single sensor-based methods. The results showed that the MHE based approach gives more accurate estimates of the robot's velocity and position with reduced noise. However, the research highlights the need for future research to address the state estimation in more complex uneven terrains with the use of additional foot force sensor data [20]. Another study uses legged robots to perform locomotion over soft and rigid terrain. The author uses state-of-the-art state estimator that combines IMU readings and leg odometry data to discover how terrain impacts the state estimation of a robot. Tests were conducted on soft and rigid terrains to analyse the differences of the state estimation. The results confirmed that soft terrain gives larger errors in the position estimates and the velocity, and it confirms the original hypothesis. Moreover, the study suggests that future work should focus on extending the state estimator to focus on the terrain impedance in the leg odometry [21].

Similarly, research was conducted using a quadruped robot to evaluate the performances of the Extended Kalman Filter (EKF) and the Unscented Kalman Filter (UKF) in the state estimation. The robot was tested on the UKF simulation framework by performing trajectories on both flat and rough terrain. The results showed that UKF outperformed the EKF, reducing final position error 30% on flat terrain and 50% on rough terrain. The paper suggests using more detailed models and proper parameter tuning can achieve further enhancements [10] .

There is another study conducted using both the Gazebo simulation and the real-world Unitree GO2 robot[22] . The paper introduces the Dual β -Kalman Filter as a reliable state estimator for legged robots that can handle foot slippage. The author tested the Dual β -KF algorithm in both Gazebo simulation and on the physical Unitree GO2 robot. The experiments demonstrated that the proposed method improved the estimation accuracy compared to the other models (UKF and QEKF) and β -KF mitigated the impact of foot slippage. The paper highlights future work that should improve the model by making β an adaptive parameter, enabling the model to respond to dynamically changing environments.

Building on this, another research about state estimation on unstable and slippery terrain was conducted on a legged robot called StarlETH. This study also uses UKF to fuse information from the robot's kinematics. The filter was tested on the robot throttling over slippery terrain. The results indicate that the filter's estimates for roll and pitch were very precise, with RMS value below 0.01 rad. The authors suggest that future work should consider integrating other different sensor modalities for navigation and terrain perception [23].

Another research was conducted to present a biped robot state estimation system that operates without external sensors, using soft margin Support Vector Machines for foot ground contact detection and EKF to fuse the kinematics and the IMU data. Training the SVM is done with hip and knee joint torque measurements, labelled with readings from temporary pressure sensors that allow robust contact detection without fragile force sensors or precise dynamic models. Experiments were conducted on a 3D biped robot, comparing the SVM contact detection method to a torque threshold method and testing state estimation results using kinematic-based, IMU-based, and fused data. SVM-based contact detection achieves higher accuracy compared to the torque method. EKF fusion shows short-term accuracy on IMU-based estimation [24].

Similarly, another study aims to improve state estimation in mobile robots by using a 1:4 scale prototype with differential GNSS sensors, incremental wheel encoders, and an absolute encoder on steering. This is designed for container transport in a port environment. The researchers suggest a new method of sensor fusion to produce better results where a nonlinear observer (NLO) is coupled with an unscented Kalman filter whose parameters are globally

optimised by a practical swarm algorithm to reduce root mean square error. The method was tested by MATLAB simulations using the Monte Carlo trials and real-world experiments, and the performance of the method was compared with the EKF, UKF, CKF, HUKF, and standalone NLO methods. It was found that the NLO-UKF algorithm gives the lowest root mean square errors in position and orientation, compared to the DGNNS at accuracy of 1 cm, and computations were much faster than other higher order UKF variants. Nevertheless, the approach is prone to GNSS signal failures and environmental noise, so future research may want to combine more sensors like cameras or radars to improve localization quality [25].

Cvitanic et al. [26] focuses on the enhancement of the robot end-effector state estimation in industrial atmosphere based on a laser tracker and inertial sensor fusion. The authors use Kalman and Particle Filters to integrate laser tracker position/ orientation data with IMU acceleration and angular velocity to accurately estimate the full state, which consists of velocity and angular acceleration. The experiments were performed on MATLAB simulations of 3-axis and 5-axis motions with varying sensor noise and hardware implementation on a Kuka KR 500-3 manipulator. They resulted in up to 95% improvement in the accuracy of velocity estimation and up to 45% improvement in the accuracy of angular acceleration compared with individual sensor applications. The research gap is in expanding sensor fusion to closed-loop control in velocity-sensitive systems and solving issues such as calibration errors and real-time capability in more demanding industrial work [26].

2.2 Learning-Based State Estimation Methods

In line with the previous studies, this paper investigates proprioceptive state estimation for legged robots using a neural measurement network (NMN) fused with an invariant extended Kalman filter (IEKF). The proposed method is tested on a Raibo quadruped with IMU and joint encoder, with Vicon motion capture as the ground truth on various types of surfaces such as flat, debris, slippery, and soft terrains. Foot contact probabilities and body-frame velocities from NMN and GRU-MLP are fused through the IEKF with training to reduce the gap between real life and simulation. Experiments demonstrate a consistent decrease in drift with use of IEKF + NMN, resulting in better position accuracy, and smoothness loss improves stability, and domain randomization, in most cases, proves to decrease performance. The above results

were solely on simulated data, and future research can incorporate real-world data with different estimators [27].

Albadin et al. [28] uses a model-free approach to determine leg height and Ground Reaction Force in mobile robots, based upon a Bidirectional Long Short-Term Memory (Bi-LSTM) network, without using foot-mounted force sensors. A simulated hexapod robot was tested under normal and damaged-leg conditions and verified on real data on the MIT Mini Cheetah, with joint encoder, torque, and IMU measurements. The network was optimized for sensor selection, sequence length, and computational efficiency and was compared with other networks, which include Stacked LSTM, MLP, and Invariant Extended Kalman Filter. The results show high accuracy, up to 99% R^2 in simulation and up to 92% on real data, demonstrating robustness even with the damaged legs, and computational time is reduced 17% with the use of only joint encoders. The authors point out that future work requires further real-world experiments for hexapod robots on difficult terrain and propose to apply the methods to fault diagnosis, failure adaptation, and safe operations of robots [28].

Zhuangwei Shi [29] examines state estimation for a simulated linear mobile robot model using Python and PyTorch. It suggests an implementation of LSTM in combination with Transformer and the EM-Kalman Filter within a sequence-to-sequence encoder-decoder framework, forming LSTM-KF, Transformer-KF, and TL-KF models. Experiments were done to test these models by estimating unknown system parameters and comparing their results to EM-KF on synthetic robot motion data. Results showed that EM-KF estimated observation noise well, Transformer-KF was better for process noise, LSTM-KF worked best for initial states, and the combined TL-KF gave the most accurate and robust results overall. Nonetheless, it was applied to linear systems only, and the future research must expand the approach to nonlinear systems and consider other more advanced models including CNNs, GANs, or bidirectional transformers [29].

Lima et al. [30] explore an end-to-end deep learning model of 6-degree-of-freedom (6-DOF) odometry with low-cost inertial measurement units (IMUs) on handheld devices and micro aerial vehicles and test it on the OxIOD dataset and the EuRoC MAV dataset. The proposed method integrates convolutional neural networks with a two-layer bidirectional LSTM and

employs multi-task loss balancing based on translation mean absolute error and quaternion multiplicative error. The experiments included training and evaluation on a wide variety of motion sequences, pose representation, and loss evaluation using both qualitative trajectory visualization and quantitative metrics like RMSE and MAE. It is shown that the translation-vector plus quaternion representation produces better trajectory accuracy and much less drift compared to state-of-the-art inertial odometry. Nevertheless, the method also has problems with the accumulation of errors in long sequences and dependence on variations in users and devices as well as motion models, which will be addressed in future improvements including periodic visual feedback, adaptation to the domain, and zero-velocity constraints [30].

A separate study uses a 3-DOF robotic leg with IMU accelerometers and joint encoders by obtaining ground-truth data from a foot-mounted force sensor. Instead of relying on noisy or failure-prone force sensors, the researchers transformed time-series sensor signals into phase images through Stacked Channels Image (SCI) and Recurrence Plot (RP) encoding, which were then classified using lightweight deep neural networks such as EfficientNet-B0, ShuffleNet V2, and RNNs. The models were trained on 135,997 labelled stance/flight samples, tested in scenarios such as hopping, falling, side impacts, and variable ground height, and validated using an independent dataset of MIT Mini Cheetah along the eight terrains. The experiments demonstrated that using EfficientNet-B0 as a backbone model and RP encoding yielded a near-perfect classification accuracy rate of 98.97 percent on the robotic leg dataset and 96.51 percent on the MIT Mini Cheetah dataset, with ONNX optimization reducing inference time to approximately 2 ms, making it suitable to apply in real-time scenarios. Nevertheless, the attained performance can be further enhanced by adding new types of sensors, optimizing multi-sensor fusion rules, and investigating superior models like transformers to further generalize across a broad variety of robots [31].

Similarly, a study presents a CNN based relocalization method for indoor mobile to address the kidnapped robot problem. The system employs the encoder-decoder network of ResNet18 and a new planar state parameterization that simplifies the system by focusing on the 3-DOF motion. Experiments were carried out on the OpenLORIS-Scene dataset using cafe and corridor sequences, and results were compared with PoseNet and PoseLSTM. The proposed approach was found to be equally accurate in translation as PoseLSTM, but with higher

orientation accuracy compared with both baselines. However, the orientation representation depending on Euler angle variants can have periodicity problems, and the authors recommend an exploration of viewpoint parameterization and a combination of the method with SLAM to achieve an exact localization in the future [32].

Sojka et al. [33] employ a quadruped robot called ANYmal B300 with force/torque sensors in its feet to measure haptic data when walking over uneven terrains in both controlled and complex environments. The authors present a transformer-based neural network called Signal Transformer, trained with triplet loss to generate compact and discriminative embeddings from raw haptic signals, which are then integrated with the Monte Carlo Localization framework. Experiments were done on the PUTany dataset, where robot trials were run over 3.5m x 7m space consisting of eight different surface types, showing the approach was able to achieve better performance than the state-of-the-art approaches. The results greatly minimized localization errors by achieving up to 50% accuracy compared to the prior methods, and it is also 10 times faster in terms of inference time and less memory via embedding compression. However, the study reveals that more validation is required to be done in large scale outdoor areas with less clear terrain boundaries, which will be an area of future development [33].

2.3 Hybrid Sensor Fusion Approaches

A study introduces hybrid state estimation for legged robots called OptiState. It was tested on the robot called SCALER, using IMU, joint encoders, a depth camera, and motion capture. The system combines a Kalman filter and a GRU network to extract Vision Transformer features from depth images. Experiments were conducted by training GRU offline using 16 trajectories and comparing it to VIO SLAM and ablated versions. OptiState achieved an average 65% RMSE improvement over VIO SLAM. Furthermore, the study suggests, the future research should focus on methods that allow the system to generalize to unknown larger environments [34].

In response to the above findings, research about autonomous driving uses Nuscenes CAN bus dataset with added sensor failures in IMU, steering encoder, wheel encoders, and noisy GNSS to simulate real-world issues. It uses parallel LSTMs to learn the dynamics of each sensor,

implements a selective feature fusion approach, and updates predictions on a Kalman Filter like structure without any explicit noise assumptions. The models are trained on 900 scenes and tested on 100 scenes, with and without failures. The result shows the framework outperforms a kinematic model, extended Kalman filter, and KalmanNet in robustness and accuracy. It achieves low errors in position, yaw, velocity, and yaw rate under multiple simulated sensor failures. The future improvements should include learning the measurement model and extending to richer state representation of ego motion for further robustness [35] .

Gu et al. [36] uses Deep learning Kalman Filter to enhance the leg odometry of quadruped robot through learning and compensation of the foot contact bias. With the help of a Unitree Go1 robot including IMU, ground reaction force sensors and LiDAR, an LSTM-based recurrent neural network has been trained to predict contact bias and incorporate it into a Kalman filter model. The evaluation of the method was based on comprehensive simulations of flat, uneven, sloped, stair, block, and hilly terrains and real-world experiments, indicating an average 64.93% translation errors reduction relative to conventional Kalman filter odometry. The DLKF additionally enhanced localization precision, especially when combined with LiDAR odometry, especially in feature-sparse environments. Nevertheless, due to the restricted range of experimental training data in Flat terrain, the generalization of the model is limited meaning that additional terrain data should be applied to enhance its strength in the real-world setting [36].

Zhao et al. [37] use a quadruped robot dog equipped with an Intel RealSense D435 stereo camera and a 3DM-GX3-25 IMU, running on an Ubuntu laptop with ROS. The researchers have created a pose estimation system relying on an Error-State Kalman Filter (ESKF) that combines internal sensor data (joint encoder + IMU) with external vision data of ORB-SLAM3. They initially tested the algorithms with EuRoC data, and then test with a real-world rectangular trajectory, and compared internal-only estimation with and without IMU angular velocity compensation against the fused approach. The fusion scheme minimized endpoint error by approximately 73.5 percent in the event that IMU compensation was not present, demonstrating greater accuracy and strength. However, the fusion did not contribute much to IMU compensation when it was already accurate and suggested the necessity of adaptive fusion strategies or superior vision estimation in high-quality IMU [37].

Liu et al. [15] use Unitree Go1 quadruped robot equipped with a body IMU, four foot-mounted IMUs, joint encoders, and foot contact sensors, with mocap data used only for offline training and validation. The authors extend the concept of the right-invariant extended Kalman filter (RI-EKF) to a multi-IMU design and combine it with a one-dimensional ResNet model which predicts or corrects the yaw angles, avoiding the use of external vision systems. The experiments were conducted on the real and noise-augmented datasets to compare the single-IMU odometry, multi-IMU odometry, RI-EKF, and the proposed learning-assisted multi-IMU proprioceptive state estimation (LMIPSE), wherein accuracy was measured using mocap ground truth. LMIPSE achieved lower drift and root-mean-square errors than traditional EKF or RI-EKF methods using only IMUs, providing performance close to motion-capture-assisted methods while meeting real-time constraints. However, future directions will be to validate the approach in outdoor, irregular-terrain settings, further generalize datasets, and investigate learning-based methods to optimize filter hyperparameters [15].

2.4 Visual and Exteroceptive Sensor Integration

A different study introduces BIG-STEP, an improved state estimator for the legged robots, which was tested on the Unitree Go1 quadruped robot with a stereo camera, IMU, and LiDAR that can operate in dynamic environments like paved roads, stairs, and hills. The approach builds upon the existing STEP VIO pipeline by adding a fast and robust ground segmentation algorithm, based on a tri-grid field representation. Experiments were conducted by comparing VINS-Fusion and STEP. BIG-STEP and STEP showed better results in estimating the states of the robot with significant improvement in the z-component. Furthermore, improvements mentioned by the authors include introducing a ground factor into the factor graph to couple tighter and creating adaptive ground-feature recognition or fusion with LiDAR to gain more robust segmentation [38].

Kang et al. [39] a decentralized state estimation system for legged robots, implemented on the platforms, including hopping robot PogoX, bipedal robot Casie, and quadrupedal Unitree Go1, with use of onboard IMUs, encoders, cameras, and LiDAR sensors. The approach combines orientation estimation using an Extended Kalman Filter (EKF) with a velocity estimation using a Moving Horizon Estimation (MHE) that allows efficient estimation fusion between

proprioceptive and exteroceptive sensors and estimates with state constraints. The ROS2 implementations have been tested in both indoor and outdoor environments with a comparison to OpenVINS and Invariant EKF. It was found that the framework was able to provide high-frequency (200 Hz) and precise state estimation, surpassing prior methods with less RMSE in velocity and orientation, as well as robustness to dynamic locomotion tasks. The authors indicate that the future directions include enhancing computational efficiency of quadratic programming, comparing arrival cost formulations, and extending the methodology to contact force, joint torque, and extended state estimation [14].

Agrawal et al. [40] targets legged robots like Unitree A1, ANYbotics Anymal C robots, and the Atlas by Boston Dynamics that need proper state estimation in rugged terrain where exteroceptive sensors such as cameras and LiDAR tend to fail. The authors introduce a factor-graph based architecture, which utilizes IMU pre-integration, joint encoder measurements, and contact detection in the kinematic chain to obtain the estimates of both the base and foot states, relying on primarily proprioceptive data. Experiments were conducted in simulation and on real hardware using different trajectories and were compared to a widely used baseline estimator. Their approach had an average improvement of 27% in accuracy in Absolute Pose Error and Relative Pose Error, showing improved drift management and cross-platform performance. The future work suggests that force-torque measurements and slip estimation can be used to make the work even more robust and more practical [40].

2.5 Multi-Platform and Modular State Estimation Frameworks

Nistico et al. [7] experimented on two quadruped robots (Aliengo and ANYmal B300) mounted with IMUs, joint encoders, cameras, LiDARs and torque sensors to test the MUSE state estimation architecture in a laboratory setup and in outdoor environments including slippery and uneven terrains. The framework uses a modular pipeline to fuse the proprioceptive and exteroceptive information including slip detection, leg odometry, attitude observers, and Kalman filter based sensor fusion to provide robust pose and motion estimates. Experiments have been performed including closed loop locomotion with the Aliengo, as well as benchmarking on the Fire Service College dataset against state-of-the-art estimators including Pronto, VILENS, TSIF, and DLIO using absolute trajectory error and relative pose error

metrics. Results show that MUSE greatly enhances accuracy with up to 67.6% increase over Pronto and up to 26.7% improvement over VILENS with high operational frequencies (up to 1000 Hz) and trustworthy feedback for real-time control. Future work should target refinements of contact modelling, better slip detection by probabilistic or learning methods, and consider dynamic contact point variations [7].

Similarly, Nistico et al. [41] analyse state estimation schemes for quadruped robot with the KAIST HOUND2 platform in the indoor and outdoor environments using IMU, joint encoders, LiDAR, and GPS. The authors suggest two methods, the Extended Invariant Extended Kalman Filter (E-InEKF) and the Extended Invariant Smoother (E-IS), which integrate proprioceptive and exteroceptive measurements using group-affine observation models to reduce the position drift. They tested the algorithms in indoor walking experiments on non-flat slippery terrain and in outdoor long-range (300m) closed loop experiments and compared them to baselines such as LIO-SAM, FAST-LIO2, P-InEKF and P-IS. The suggested methods achieved a reduction of up to 28% Absolute Trajectory Error (ATF) indoors and 40% ATF outdoors compared to LiDAR-based baselines, where E-IS achieved better accuracy while E-InEKF provided better real-time performance. The authors state that future work could be done to add more modalities, such as RGB-D and thermal cameras, and investigate tightly coupled fusion schemes to better optimize the accuracy-efficiency trade-off [41].

A summary of the reviewed literature is presented in Table 1.

Table 1: Summary of the Literature Review

Study / Method	Sensors Used	Terrain Type	Architecture / Approach	Key Results	Limitations	Relevance / Critique
Nistico et al. 2025 (MUSE) [7]	IMU, camera, LiDAR, encoders	Indoor and outdoor	Multi-sensor Kalman fusion	Improved trajectory accuracy by 67.6 percent over Pronto and 26.7 percent over VILENS in	No stereo vision	Closest to field benchmark for this study

				real-time at 1000 hertz.		
Gu et al. 2024 (DL-KF) [36]	IMU, LiDAR, force	Flat and hilly	LSTM + Kalman Filter	Reduced translation error by 64.93 percent compared with standard Kalman Filter on flat and hilly terrain.	Terrain-limited	Promising learning fusion for outdoor use
Kang et al. 2024 (EKF + MHE) [39]	IMU, LiDAR, camera	Indoor and outdoor	Decentralised fusion	Achieved 200 hertz real-time fusion with lower velocity and orientation RMSE compared with OpenVINS and IEKF.	Complex optimisation	Good baseline for multi-sensor bush applications
Youm et al. 2024 (NMN + IEKF) [27] (Dual β -KF)	IMU, joint encoders	Simulated flat and debris	GRU-MLP + Invariant EKF	Less drift, better position accuracy and stability on simulated flat, debris, slippery and soft surfaces.	Simulation only	Relevant for terrain variation but needs real - world validation

Vecchione & Boaventura 2025 (UKF) [10]	IMU, encoders	Flat and rough	Unscented Kalman Filter	UKF minimized final position error by approximately 30 percent on flat and 50 percent on rough surface as compared to EKF.	Requires fine tuning	Strong benchmark for uneven terrain but works on known models only
Nisticò et al. 2025 (E-InEKF / E-IS) [41]	IMU, LiDAR, GPS, encoders	Indoor and outdoor	Invariant Filter + Smoother	Reduced absolute trajectory error between 28 and 40 percent compared with traditional IEKF.	No RGB-D Fusion	Promising fusion, moderate relevance to bush terrain
Song et al. 2023 (BIG-STEP) [38]	IMU, LiDAR, stereo camera	Stairs and hills	Segmentation-based VIO	Large accuracy improvement in vertical axis over VINS-Fusion and STEP.	No learning fusion	Geometric only, limited to built terrain
Schperberg et al. 2024 (OptiState) [34]	IMU, encoders, depth camera	Indoor	GRU + Transformer + Kalman Filter	GRU + Kalman yields smoother trajectories and reduced RMSE than VIO which	Small dataset	Strong hybrid idea, indoor only

				is qualitatively better.		
Zhao et al. 2022 (ESKF + ORB-SLAM3) [37]	IMU, encoders, stereo camera	Rectangular course	Error-State Kalman Filter + Visual SLAM	Reduced endpoint error by about 73.5 percent using IMU-vision fusion compared with vision-only baseline.	Limited IMU gain	Good fusion study, short tracks only. Need long-terrain trials
Zhang et al. 2025 (Dual β -KF)	IMU, encoders	Slippery	Dual β -Kalman Filter	Dual β -KF was more accurate than the UKF and QEKF and reduced the effects of foot-slip.	Static β , not adaptive	Good for slippage cases only not for general terrain
Albadin et al. 2025 (Bi-LSTM) [28]	IMU, encoders	Normal and damaged legs	Bi-LSTM state predictor	Coefficient of determination R^2 equal to 0.99 in simulation and 0.92 in real data; computation time decreased by 17 percent using only encoders.	Needs outdoor tests	Robust proprioception, no visual fusion

Agrawal et al. 2022 (Factor Graph fusion) [40]	IMU, encoders, contact sensors	Indoor and outdoor	Kinematic chain factor graph	27% better pose accuracy	No vision	Good proprioceptive reference
Fahmi et al. 2021 [21]	IMU, leg odometry	Soft and rigid	Classical fusion filter	Impact of terrain-impedance was confirmed by large position and velocity errors on soft terrain compared to rigid terrain.	No terrain adaptation	Highlights sensitivity to unstructured ground And outdated
Shi 2021 (LSTM / Transformer-KF) [29]	Synthetic IMU	Simulated	LSTM + Transformer KF	Transformer-LSTM Kalman Filter was the most accurate and robust on synthetic motion data.	Linear systems only	Good sequence model, no robot tests
Liu et al. 2025 (LMIPSE) [15]	Body and foot IMUs, encoders	Flat indoor	1D ResNet + Invariant EKF	Lower drift and better trajectory continuity than standard EKF under the same conditions.	No rugged tests	Strong baseline, lacks terrain diversity
Dahal et al. 2024 (RobustStateNet) [35]	IMU, GNSS, wheel encoders	Autonomous driving scenes	LSTM fusion network	Resilient under sensor failure.	Vehicle domain	Good robustness idea for noise.

Sojka et al. 2022 (Signal Transformer) [33]	Force/ torque sensors	Uneven indoor	Transformer haptic localization	50% better localization, 10X faster	Need outdoor validation	Innovative terrain encoding
Yazdipaz et al. 2025 (Signal Imaging + DNN) [31]	IMU, encoders	Various terrains	SCI/RP + EfficientNet	Accuracy 99% stance / flight	Limited sensor fusion	Strong for gait and contact
Gao et al. 2021 (SVM + EKF) [24]	IMU, torque, encoders	Flat lab	SVM contact detection + EKF	SVM contact detection more accurate than torque threshold. EKF fusion improved short-term IMU accuracy.	Biped only	Method does not scale to quadrupeds
Yu et al. 2020 (MHE) [20]	IMU, leg kinematics	Simulated flat	Moving Horizon Estimation fusion	MHE fusion generated less noise velocity and position estimates as compared to single sensor techniques.	Not tested on uneven terrain	Accurate in lab but not generalizable to natural bush
Bloesch et al. 2013 (EKF) [23]	IMU, leg kinematics	Slippery floor	Extended Kalman Filter	Roll and pitch RMS of less than 0.01 radians on slippery surface.	Limited sensor inputs	Baseline filter, fails under non-linearity
Cvitanic et al. 2022 [26]	Laser + IMU	Industrial	Kalman + Particle Filter	Velocity accuracy increased by 95 percent and	High latency	Industrial setup, no

				angular acceleration by 45 percent compared with single-sensor configuration.		mobility testing
Kuncara et al. 2024 (NLO-UKF) [25]	GNSS, encoders	Port area	Non-linear Observer + UKF	Mean square error of about 1 centimetre with fastest computation among UKF variants.	Fails under GNSS loss	Not usable in GNSS-denied bush zones
Lima et al. 2019 (CNN+BiLSTM)	IMU	Structured indoor (handheld, MAV)	CNN + 2 layer Bi-LSTM with multi task loss	Better trajectory accuracy and less drift vs prior IMU odometry	Long – sequence drift, no terrain or robot tests	Conceptually supports learning – based 6-DOF odometry, but low terrain or modality relevance
Yang et al. 2021 (CNN Relocalization)	RGB Cameras	Indoor	ResNet-18 encoder decoder, planar 3-DOF CNN relocalization	Matched PoseLSTM translation accuracy, better orientation accuracy	Limited to Euler angles, no 6-DoF fusion or terrain data	Relevant for learning based relocalization but not terrain or multi-sensor fusion context

3.0 Research Overview

3.1 Research Gap

Quadruped robot visual odometry has advanced in controllable and semi-structured environments but has not been applied to rugged bush environments. Photorealistic simulators such as NVIDIA Isaac Sim, and Unity allow modeling of quadruped robots, including the Unitree Go2 [42], [43]. Platforms including Gazebo, PyBullet, MuJoCo and Webots allow quicker prototyping and ROS2 integration, but they are less powerful platforms [44]. Despite these tools, no particular simulation framework had been designed to simulate unstructured bush environments with quadruped odometry and this work addresses that gap by developing such framework.

Quadruped datasets in the public are primarily concerned with proprioceptive sensors, IMU, and joint encoders, including the HyQ dataset [45]. There are also multi sensor datasets with cameras, IMU, LiDAR, and ground truth information of ground robots available [46], [47]. Examples of multi-sensor datasets include the Oxford Multimotion Dataset [48], the FusionPortableV2 Dataset [49], and the Rosario Dataset [50]. There are no equivalent datasets for rugged bush or forest; there is a gap in current simulation environments with realistic vegetation assets and there is a lack of custom datasets with quadrupeds, like the Unitree Go2, in rugged bush environments. This gap limits benchmarking and reproducibility in rugged terrain, and this research narrows it by producing a simulation environment with realistic vegetation assets and a custom quadruped (Unitree Go2) dataset generated within it.

Sensor fusion of quadrupeds has been directed more towards vision and inertial sensors. As an example, MUSE integrates several sensors, such as IMU, encoders, cameras, and LiDAR in real-time but does not make use of stereo vision or deep learning techniques [7]. Similarly, VILENS uses vision, IMU, LiDAR, and leg odometry on a factor graph structure, but also does not involve stereo camera information and does not delve into neural network fusion [51]. Combining stereo cameras, IMU, magnetometers and joint states into state estimation with quadrupeds in unstructured natural terrain has not been methodically investigated and this research addresses that gap directly.

Although neural networks have been proposed in odometry, comparative analysis in terms of architecture choice has been limited. 3D CNNs (Video ResNet) [52] are capable of obtaining spatio-temporal features of short sequences and are also efficient on smaller datasets [53]. Transformers (Video MViT) [54] have the ability to model longer temporal dependencies and more flexible multimodal fusion at the cost of larger training data and regularization [55], [56]. The relative advantages and disadvantages of both methodologies had not been examined for quadrupeds in rough bush terrain, which this work addresses.

Beyond choosing a specific network architecture, how distinct sensor types are fused remains largely unstudied in a comparative way. Learning-based odometry typically relies on end-to-end mapping from raw sensor streams straight to motion estimates [30]. Conversely, classical quadruped pipelines, such as MUSE [7] and VILENS [51], fuse hand-engineered, per-modality estimates inside a filter or factor graph. These two paradigms are rarely evaluated side-by-side on identical hardware and terrain. As a result, it is still unclear whether a quadruped navigating rugged bush performs better by learning fusion end-to-end from raw inputs, or by preprocessing each modality independently (e.g., using an inertial network for the IMU and magnetometer, forward kinematics for the legs, and visual SLAM for stereo cameras [57]) before fusing those features. It is equally unestablished which temporal architecture works best once inputs are preprocessed this way. This work addresses both gaps directly, comparing end-to-end learning against a preprocessing-and-fusion framework while benchmarking multiple architectures within it.

3.2 Research Questions

The key research question is:

1. Does a 3D CNN (Video ResNet) [52] or a transformer (Video MViT) [54] perform quadruped state (6 Degree of Freedom Velocity) estimation more accurately, when traversing a bush environment using an IMU, magnetometer, stereo camera and joint states?

The secondary research questions are:

2. When using stereo cameras for quadruped state estimation, does inputting RGB images or preprocessed stereo disparity produce more accurate results and what is the difference?
3. Does a state estimation neural network for a quadruped- using an IMU, magnetometer, joint states and stereo-cameras as input- trained in simulation (with real world data input and domain randomisation) transfer to equivalent accuracy in the real world? If not, what is the accuracy difference?
4. Is state estimation more accurate with an architecture that is trained end-to-end learning or by preprocessing the sensors input before sensor fusion (using neural networks for the IMU and magnetometer, joint odometry for the legs and SLAM for the stereo-cameras)?
5. Which architecture produces the most accurate state estimation when preprocessing sensor inputs?

4.0 Methodology

The research utilized a simulation-based (Isaac Sim) [58] experimental framework to train and test machine learning algorithms to estimate the states, which were the linear and angular velocities of the origin of a quadruped, followed by an initial test of the trained models on real-world data (Sections 4.9 and 5.7). The methodology included dataset creation in simulation followed by experiments using end to end learning of state estimation across different neural network architectures. When it was found that performance of end to end learning was poorer than expected on large scale datasets, the research pivoted towards experimenting with different preprocessing methods for the sensor inputs into a range of different neural networks.

4.1 Data Collection in Simulation

Training data was gathered in simulation with NVIDIA Isaac Sim and a Unitree Go2 quadruped model [58]. Each data collection episode consisted of the quadruped traversing a range of environments, while simulated sensor data was collected. The simulation was run in headless mode to allow data to be collected automatically at higher speeds. The robot locomotion was generated by a hand-tuned trot gait with a frequency of 2.0 Hz, thigh amplitude 0.15 rad and calf amplitude 0.35 rad. The joint targets were controlled using a PD controller with $K_p = 80.0$ and $K_d = 2.0$ to maintain a stable posture at a body height of approximately 0.28 m. The physics engine was set to 60 Hz. Proprioceptive sensors (IMU, magnetometer, and joint states) were sampled at each physics step, yielding a data rate of 60 Hz, while stereo camera images were captured at 30 Hz. Each trajectory was about 60 seconds long, resulting in about 3,600 frames.

At every frame, the following data were recorded:

- Stereo RGB images from left and right cameras at 1280×720 resolution.
- A six-dimensional IMU vector, with linear acceleration and angular velocity along x, y, z axes.
- A three-dimensional magnetometer vector.
- 24-dimensional leg joint state vector, with 12 angles and 12 angular velocities.

- The ground truth labels, which provided six-degree-of-freedom velocity in the robot body frame, with linear components v_x, v_y, v_z and angular components $\omega_x, \omega_y, \omega_z$.

4.1.1 Terrain Environments

Data collection included five types of terrain: flat, forest, rugged, uphill, and downhill slopes. A three-staged curriculum was used to select terrain. This provided more exposure of forest, which was the planned deployment environment at Phyllis Reserve.

Stage 1, Forest-Flat: Approximately 30% of the total trajectories were conducted in Stage 1. The environment was sampled out of a pool of forest and flat terrain choices, resulting in a distribution of about two-thirds forest and one-third flat ground. During this initial stage, the surfaces remained uniform, and no additional terrain challenges or irregularities were introduced.

Stage 2, Forest-Mixed: The second stage comprised 40% of the trajectories and introduced a more diverse terrain mix. The sampling pool expanded to include forest, flat, and uneven surfaces. To simulate real-world navigation challenges, rough terrain was configured with a 0.05 m roughness factor, while sloped sections featured a 5° incline.

Stage 3, Forest-Slopes: The final 30% of the data collection occurred in Stage 3, representing the highest level of difficulty. This stage utilized a selection of forest, rough, uphill, and downhill environments. Environmental complexity was significantly scaled up, with roughness parameters doubled to 0.10 m and slope gradients increased to a 10° angle.

Each forest scene was generated separately for every trajectory. Poisson-disk rejection sampling was used to avoid trees being clustered too closely. There were approximately 150 trees in each scene, and tree geometry was randomized. Tree trunk height varied from 1.5 m to 4.5 m, and trunk radius ranged from 0.04 m to 0.18 m. Canopy radius ranged from 0.60 m to 2.00 m, and canopy height ranged from 1.20 m to 3.00 m.

Tree colors also varied. Tree trunks were selected from five brown shades. Canopies were selected from seven colour variants, including green, dark green, bright green, blue-green, orange, yellow, and rust-red. Tree trunks used collision geometry, whereas canopies did not. A

4.6 m wide corridor was left open for the robot to pass through. Ten fixed landmark objects (cube, sphere, cylinder, cone, capsule, pyramid, triangular prism, diamond, torus, hexagonal pillar) were placed in each scene to provide stable visual references. A further eight primitive objects were added and randomized each trajectory.

4.1.2 Domain Randomization

Domain randomization[59] was used to increase the variability of the scene and avoid overfitting to a particular scene or lighting condition.

Forest layout: The seed controlling the placement, size, and colour of the forest trees was drawn separately for each trajectory (seed drawn uniformly from $[0, 2^{31}]$) so that each forest trajectory had a different tree arrangement.

Spawn position and heading: For forest terrain, the robot spawned at a random position with $x \in [0, 2]$ m and $y \in [-1.5, 1.5]$ m. For the other terrain types the spawn was within ± 1.5 m of the origin in both x and y . The initial heading was uniformly drawn from $\pm 20^\circ$ (± 0.35 rad) around the forward direction.

Trajectory length: The number of frames in each trajectory was multiplied by a uniform random number from the range $[0.85, 1.15]$, causing $\pm 15\%$ variation around the nominal 3,600 frames per trajectory.

Path type: The robot's path was randomized at the beginning of each trajectory to be either a forward or a circular arc.

Object Appearance: The eight scene objects were fully randomised per trajectory using a DomainRandomizer module: size was scaled by a factor from $[0.25, 4.0]$; per-axis stretch factors from $[0.5, 2.0]$ were applied on top to produce non-uniform shapes; mesh vertices were displaced by a crop factor from $[0.0, 0.5]$; and colour was assigned as a random HSV material with hue from the full spectrum, saturation from $[0.5, 1.0]$, and value from $[0.5, 1.0]$.

Lighting: Scene illumination was randomised per trajectory. A dome light was set to a random intensity (between 1,000 and 100,000 lux) and colour temperature (between 3,000 and 8,000) K was used to represent lighting indoors or outdoors. With probability 0.5 a directional light was added with a randomly sampled direction.

Sensor Noise: Gaussian noise was added independently to three sensor streams at each simulation step: accelerometer ($\sigma = 0.02 \text{ m/s}^2$), gyroscope ($\sigma = 0.002 \text{ rad/s}$), and joint position targets ($\sigma = 0.01 \text{ rad}$).

Ground friction: Physics materials with static and dynamic friction 1.0 and 0.8 (higher than the Isaac Sim default of 0.5) were applied to terrain ground planes to simulate foot friction.

Figure 1 shows simulated forest like environment.



Figure 1: Simulated forest like environments

4.1.3 Dataset Statistics and Filtering

Data collection was conducted headlessly for roughly 3.5 days, with 1,081 trajectories spanning some 3.72 million frames of video and 1.5 TB of raw stereo imagery.

Falls were automatically detected via robot body height: if height fell below 0.10 m (with 30-frame grace period), a FALL marker file was written and trajectory collection stopped. A total of 73 (6.7 %) of the 1,081 trajectories were detected as falls and removed; this included all 20 downhill trajectories. The remaining 1,008 "clean" trajectories were distributed across terrain as follows: flat (284), forest (627), uneven (90) and up hill (7).

Seven trajectories had angular velocity label spikes of 150-188 rad/s, which are impossible for the Go2, and may be due to sensor glitches or missed falls. Angular velocity labels in the entire dataset were capped at ± 10 rad/s during training to retain valid motion data while eliminating the bias on mean-squared-error loss.

4.2 Dataset Split

The 1,008 clean driving trajectories are divided into training (650), validation (150), and test (208) trajectories. The trajectories are split at the trajectory level rather than the frame level within each trajectory. The frames within each trajectory are temporally correlated with one another; splitting the dataset at the frame level would lead to model observations of nearly identical frames within both the validation and test split, which would lead to poor validation and testing metrics.

The initial random split of the data did not include any trajectories with high angular velocities in the validation split. The loss of the validation split was approximately 11 times lower than the loss within the training split, making its validation metrics meaningless. To avoid this issue, the data is split by terrain type (flat, forest, uneven, uphill) using stratified sampling. This assures that each split contains an equally represented sample of each terrain type and difficulty. A random seed of 42 is used to ensure reproducibility of the results. Trajectory counts per terrain type are shown in Table 2.

Table 2: Trajectory Counts Per Terrain Type After Stratified Splitting

Terrain	Train	Val	Test
Flat	~183	~42	~59

Forest	~404	~94	~129
Uneven	~58	~13	~19
Uphill	~5	~2	~1
Total	650	150	208

4.3 Initial End-to-End Visual Models.

The initial direction of this project was to estimate the state of the quadrupeds using raw visual inputs and sensor streams with modern deep learning architectures. Two architecture families were explored during this phase. The first architecture was a three dimensional convolutional neural network based on Video ResNet-18 (R3D-18). The second architecture was the Multiscale Vision Transformer (MViT)[60] based on a transformer. Two different visual input representations were tested for both architectures: image sequences of raw RGB images and stereo disparity maps derived from stereo image pairs. This led to 4 experimental setups: CNN RGB, CNN Disparity, MViT RGB, and MViT Disparity. Synchronized stereo image sequences, along with IMU, magnetometer and joint state data, were used in the multimodal training configurations. The models were trained to predict the 6-DOF body-frame velocity.

4.3.1 CNN RGB

The CNN RGB model took the image sequences taken from the left stereo camera as the main visual data source. The same R3D-18[61] backbone, pre-trained on the Kinetics-400 action recognition dataset [62] , was used in the visual branch. The backbone extracted spatio-temporal features directly from short RGB clips based upon three-dimensional convolutional layers [61].

The sensor branch utilized a stack of one-dimensional convolutional layers with ReLU activations and dropout regularization and temporal pooling to produce a compact temporal motion representation from IMU, magnetometer and joint state sequences. The sensor streams and image clips were synchronized at 30 Hz, with the means that every picture was recorded at the same time as a synchronized inertial and joint-state measurement. High-frequency noise

was further temporally median filtered in the pre-processing stage of sensor measurements prior to training.

All the sensor modalities were projected onto a small latent representation prior to fusion with the visual features. Mid-level sensor fusion was employed, in which the representation for the sensor was merged with the visual representation after the second residual stage[63] of the R3D-18 backbone. The fused features were then fed into the rest of the convolutional layers and the regression head to estimate linear and angular velocity of the body-frame.

Preliminary tests on a smaller subset of trajectories showed that temporal visual information can be successfully used to estimate the velocity of a quadruped. These initial experiments were promising and provided the impetus to "scale" the architecture to the entire amount of data.

Once the larger 1,008-trajectory data set was gathered, the CNN RGB experiments were repeated with a corrected data set split in the stratified data. This data set comprised ~ 3.72 million frames and 1.5 TB of stereo imagery. Prior to training, several preprocessing optimizations were added. To minimize the disk I/O cost for training, all images were resized to 224×224 from 1280×720 and then cached locally, as the name `left_224.png`. The sensor data from the CSV files was compacted into NumPy arrays to free up memory between workers of the DataLoader. The images were then resized and later all of the resized images were stored in a single HDF5 file with size of 547 GB, which reduced the overhead of the file system during training.

Since the end-to-end CNN models trained directly on image and sensor inputs failed to scale well for the larger-scale experiments, a two-stage training strategy was adopted for experiments with the larger-scale CNN RGB models. During Stage 1, the sensor branch was trained in isolation, with no visual input for the purpose of learning useful representations of motion from the IMU, magneto-meter, and joint states alone. In stage 2, the weights of the pretrained sensor branch were transferred to the entire multimodal model and the visual branch was also added to be jointly fine-tuned. This was done to avoid the visual branch being dominant in early optimisation and therefore the network not learning from the sensor streams.

Stage 2 approach showed significant improvement with the visual inputs. The first epoch itself performed very low validation loss compared to the sensor-only baseline. This indicates pretrained sensor features with visual branch provided meaningful input to the prediction task.

Based on the observations, it was hypothesized that some meaningful motion representations could be acquired from the sensor information prior to visual fusion. This later affected the shift from the preprocessing-based multimodal fusion framework of independently processing each of the sensing modalities and then fusing them over time. The large scale CNN RGB experiments were able to get better results than previous large scale experiments, but required around five days of GPU training time and around 33.5 million trainable parameters.

4.3.2 MViT RGB

The MViT architecture[60] treats image patches as tokens to process and captures temporal and spatial interactions by means of self-attention operations. The visual branch was pretrained using a Kinetics-400 [62] model of MViT_V2_S [64]. Clips of 16 frames from RGB images were tokenized and fed to several transformer blocks. One dimensional convolutional layers were used for embedding the sensor streams independently and they were late fused with the visual representation. Unlike CNN model that fused the sensor data at the intermediate layer, the MViT model did the sensor fusion after visual encoding.

In the small-scale experiments, the MViT RGB model showed better temporal modelling ability than the CNN RGB configuration, and achieved good pilot results. Next, large scale training of MViT RGB was tested with the corrected stratified split of the data. However, CNN models needed much less computational resources than the MViT architecture. Due to the limitation of the memory size of GPU, the batch size was decreased from 16 to 4. Training time per epoch was significantly greater because of the effort of the attention operations over the visual tokens.

The model showed great promising early behaviour, but the computational expense and lack of GPUs limited the number of large-scale MViT RGB experiments that could be performed.

4.3.3 Disparity-Based Experiments

During the early stage of the research, the disparity-based visual representation was also explored along with an exploration of RGB image experiments. The goal of these experiments was to see whether depth information, as opposed to the raw RGB information, could be helpful in state estimation of the quadruped.

The left and right stereo image pairs were used to create stereo disparity maps with RAFT-Stereo [65] [66] and the pretrained checkpoint provided by SceneFlow. In the smaller pilot experiments, disparity maps were created at full 1280 x 720 resolution after a series of 32 iterations of refinement. These high-resolution disparity representations were used for the initial trajectory experiments with both CNN and MViT architectures.

The overall performance of the CNN RAFT disparity configuration was slightly less than that of the CNN RGB configuration. The MViT disparity configuration, on the other hand, achieved the best pilot performance among the initial visual configurations, and also produced the best performance in the smaller-scale evaluations, compared to the corresponding RGB-based MViT model. These observations indicated that motion cues might be more strongly conveyed in depth-oriented visual representations to a temporal learning architecture with a transformer structure.

However, for the entire 1,008 trajectory set, the cost of generating full-resolution disparity became too expensive. Therefore, the large scale disparity preprocessing pipeline was adjusted to perform half-resolution disparity generation at 640×480 resolution for increasing the processing speed and minimizing the GPU memory usage.

Despite the use of lower-resolution pre-processing, disparity generation was still a significant computational challenge. The RAFT-Stereo pipeline delivered nearly 4 trajectories/hr on a single RTX 5060 Ti unit. The average disparity generation time was about 15 minutes per trajectory, as there were about 3,600 stereo frames per trajectory. Considering the throughput observed, it was estimated that the entire data set would need about 10.5 days to be preprocessed on a single GPU before training the model could be started.

Because the generation of disparity is a very costly process and the training cost of the end-to-end visual models is already high, large-scale disparity experiments were not carried out on the complete data set. Consequently, disparity evaluation was conducted at a smaller scale and was not expanded upon after the shift to the multimodal fusion framework of a preprocessing approach.

4.4 Limitations Observed with Initial Approach

Although the initial end-to-end visual models showed promising performance on smaller pilot datasets, several major limitations were identified when the experiments were scaled to the full 1008, trajectory dataset.

4.4.1 Training Cost

One of the main challenges with the end-to-end visual learning approach was computational complexity. The large scale CNN RGB Stage 2 model took around five days to train once with the available GPU machines. The transformer-based architectures took even more time to train, because in addition to the complexity of the attention operations, they needed larger clip sizes in time.

There was an extra bottle neck for disparity preprocessing. It took about 15 minutes to generate the 640 x 480 resolution disparity with RAFT for one trajectory, which is about four trajectories per hour on a single RTX 5060 Ti GPU. To process the full data set, it would then take about 10.5 days to continuously compute with the GPUs prior to starting the model training. These costs significantly restricted the number of experiments that could be completed and made large-scale hyperparameter exploration impractical.

4.4.2 Degraded Performance on Larger Datasets

Although it was found to be performing well during the pilot experiments, this did not transfer well to the larger data set. While the smaller experiments indicated that RGB and disparity-based learning by vision were promising, both methods failed to yield a significant improvement in validation behaviour and overall accuracy when scaled to the entire set of data.

This signified that with greater diversity to the environments and complexity of the datasets, the end-to-end visual learning did not scale up well.

4.4.3 Training Instability.

Training instability was also observed during large scale experiments especially for transformer based architectures . The validation loss varied a lot between epochs and the convergence behaviour was not consistent across runs. Moreover, the GPU memory constraints and the long training time limited the extent of large-scale experimentation and hyperparameter tuning

4.4.4 Poor Scalability of End-To-End Learning

The large-scale experiments showed that the end-to-end visual learning pipeline is hard to scale efficiently for the quadruped state estimation problem studied in this work.

The models had to learn visual feature extraction, stereo depth reasoning, temporal modelling, inertial interpretation, kinematic understanding and multimodal fusion all in a single optimization process. This led to large models, long training times and poor computational scalability.

Based on these observations, a shift to modular preprocessing based multimodal fusion framework was made. In contrast to directly learning all the components from raw image sequences with a single neural network, each sensing modality was first processed with the most suitable classical estimation method and then lightweight temporal learning architectures were used for fusion. The last fusion method which was based on preprocessing reduced the number of parameters in the model from about 33.5 millions to about 455 thousands and the training time from about five days to about 1 hour and 20 minutes.

4.5 Data Preprocessing Pipeline

The final architecture did not use raw data directly as input to the learning process. Instead, each modality first passed through a classical preprocessing arm which extracted physically meaningful features before any fusion took place. This separation allowed the learned fusion

model to solve only the problem of combining already-interpreted signals rather than simultaneously learning geometry, ego-motion and orientation estimation from scratch.

4.5.1 Joint Kinematics

The raw joint encoder measurements, of the twelve joints of the Unitree Go2 were processed using forward kinematics calculated on the URDF description of the robot. The kinematic chain was implemented using the `pytorch_kinematics` library together with the official Unitree Go2 URDF description. The preprocessing pipeline estimated the foot positions and foot velocities inside the robot body frame at each timestep. To apply forward kinematics reliably two further corrections had to be made during the preprocessing phase. First, the ordering of the joints was different between the ordering that was used by Isaac Sim and the ordering that was expected by the URDF kinematic chain. The simulator joint vector was then deterministically mapped to match the ordering of the joints per leg needed by the kinematic solver. Secondly, the URDF's fixed foot joints were collapsed by the kinematics library when creating the chain. For the correct physical foot position, a manually verified feet offset of (0, 0, -0.213) meters, in each calf frame, was used in forward kinematics computation. Velocities of the feet were estimated at 30 Hz with centered finite differencing and at sequence boundaries with forward and backward differencing. The resulting foot trajectories were then used to perform body velocity estimation based on stance. Each of the four feet was given a soft contact probability. Instead of a binary contact signal, the contact state was quantified as a continuous probability function, based on the body-frame foot velocity magnitude. The probability of 1.0 was given to feet moving at or below 0.10 m/s and the probability of 0.0 was given to feet moving at or above 0.50 m/s. Values in between were linearly interpolated.

The body-frame linear velocity is estimated by taking a weighted average of the stance-foot constraints using the formula: $v_{body} = -\omega_{body} \times p_{foot} - \frac{dp_{foot}}{dt}$. Individual velocity estimates from feet in stance were integrated using weights assigned according to their contact probabilities. This process yielded a coarse leg odometry estimate for body motion, which provided a foundation for later fusion with inertial and visual measurements inside the temporal learning model.

Additionally, a calibration procedure was added to compensate for a systematic error that arose in the front-right calf joint in all experimental trajectories. Detailed analysis showed that there was a consistent offset of approximately 0.6 radians compared with the other calf joints. This difference caused the standing posture to be asymmetric and caused the foot kinematics to be distorted. To remedy this, a static correction offset was determined for each trajectory, based on the difference between the left front calf joint and the mean of the other three calf joints. It was applied during data loading without destroying the data, and before feature extraction is performed.

The resulting kinematic representation integrated foot positions, velocities, soft contact probabilities, and stance-based motion estimates. These components were organized into a structured feature vector designed for temporal fusion.

4.5.2 Inertial Navigation

Inertial data from the simulated IMU and magnetometer were processed using the Madgwick Attitude and Heading Reference System (AHRS) filter, accessed via the `imufusion` library. During the preprocessing, two inertial configurations were developed: a six axis configuration with only accelerometer and gyroscope data and a nine-axis configuration including the magnetometer observations (MARG). The filter adopted the East-North-Up (ENU) convention to meet the requirements of the body-frame orientation of the simulator. The inertial processing pipeline was tuned in particular to accommodate the special requirements for quadruped locomotion. There were acceleration spikes of about ± 4 m/s² associated with repeated foot impacts when trotting motions were used. These spikes caused the filter's default acceleration rejection too often, thus interfering with accelerometer corrections. The following were set to prevent acceleration rejection from interfering with stable roll and pitch estimation during gait transitions: Acceleration rejection threshold was set to 0° to disable the mechanism, fusion gain was set to 1.0, and the range of the gyroscope was set to 2000 dps. These changes greatly improved orientation stability for high speed body movements. The IMU-only configuration offered reliable roll and pitch information but with the absence of an external reference for heading, it drifted over time. Results of evaluation on random trajectories revealed that this configuration attained median RMSEs of about 7.9 degrees for roll and 5.7 degrees for pitch

with respect to ground truth data. The magnetometer was used in the MARG configuration, where a coordinate transformation was needed. The simulator gave magnetic field readings in a North-East-Down (NED) format and the imufusion library was in an East-North-Up (ENU) format. To remove this discrepancy and to make the coordinate systems agree, the following logic was used to remap the magnetometer measurements: $(m_y, m_x, -m_z)$

Initial experiments indicated that without intervention the library's native magnetometer fusion would result in unstable heading estimates and large yaw errors. To solve this problem, a two-pass approach to determine the orientation was adopted. The first pass was performed using the filter only for the IMU, and the stable roll angle and pitch angle were estimated. The second pass data were then used to roll and pitch compensate the magnetometer data prior to computing the yaw estimate. The resulting magnetically referenced yaw estimate was not prone to long-term drift, but had a small constant offset of $\sim 5^\circ$ due to declination.

The final product of the inertial preprocessing stage comprised of unit-norm quaternion orientation estimates and the gyroscope data in the body frame. And, it also generated the gravity-compensated acceleration for body frame and roll-pitch-yaw for subsequent multimodal fusion stages.

4.5.3 Visual Odometry

The stereo image pairs from the on-board cameras were processed in stereo mode with DROID-SLAM [57] to obtain the camera motion for each image. The original image streams were 1280×720 , but they were down-sampled to half size (640×320) before being processed. A visual odometry pipeline was used to provide the camera pose as an SE(3) [67]. The first three elements, indicating translational position, refer to the camera position relative to its starting pose, while the remaining four, representing a quaternion, describe the camera orientation. These estimates were made for each timestep, with accompanying validity flags that were used to verify tracking success. To improve processing speed, DROID-SLAM only processed every third frame (stride 3). Linear Interpolation (LERP) was used for the translations while Spherical Linear Interpolation (SLERP) [67] was used for the quaternions to build the missing intermediate frames. This way, the final path was always followed perfectly along the initial

30Hz sensor stream. Efficiency was also improved by making a fast inference configuration that avoided global backend passes for bundle adjustment. Since the data were primarily short and forward-moving without revisits, a local-window optimization was enough to obtain a good stability of the estimated values which were also more easily evaluated and notably shortened the execution time. This stride was empirically validated across flat, forest, and uneven terrain to achieve the best weighted-average trajectory accuracy across the dataset's terrain mix while running 2.5 times faster than stride 1. Initial comparisons showed that DPVO was better on flat surfaces, particularly those with low texture, but that DROID-SLAM was much more accurate in the forest. Since forest trajectories accounted for the majority of the data set, the DROID-SLAM stereo was selected as the main frontend for the multimodal fusion pipeline. Finally, roughly 1% of trajectories encountered frontend keyframe-buffer overflows. These occurrences were automatically detected and reprocessed using a greater buffer space so that the integrity of the data was maintained. The visual odometry feature vector had a dimensionality of 7.

4.5.4 Synchronisation, Normalisation, and Clip Construction

To ensure temporal consistency, the three feature streams were aligned at 30 Hz using an inner-join operation on a shared frame index preserved within each trajectory archive. Following alignment, the data underwent per-channel z-score normalisation. This process was applied to each modality independently, utilizing mean and variance statistics derived from a 20% subsample of the training trajectories. These parameters were subsequently stored with the model checkpoint to ensure consistent scaling during inference.

The final step involved partitioning the feature streams into discrete temporal windows. A fixed length of 40 consecutive timesteps was chosen for each clip, which represents roughly 1.3 seconds of physical motion at the 30 Hz capture rate. To maintain temporal context and effectively increase the training volume, a sliding window approach was implemented. This was done by using an 8-frame stride (approx. 267 milliseconds).

This overlapping strategy ensured that transitions between movements were well-represented across the dataset. Applying this method to the 650 available trajectories resulted in a total yield of 289,164 clips for training and 66,037 for validation.

4.6 Fusion Model Training

The multimodal learning stage aimed to estimate six-degree-of-freedom body motion directly from a set of synchronized sensor representations that are produced during the preprocessing stage. Instead of using only one sensing modality, the learning framework was based on an intermodal integration of proprioceptive, inertial and visual observations in a unified temporal architecture. The goal of the model was to estimate linear and angular velocity of the body-frame as: $[v_x, v_y, v_z, \omega_x, \omega_y, \omega_z]$.

Synchronized temporal clips were selected from the aligned preprocessing output for all the models. The experiments aimed to compare several temporal fusion architectures under the same training settings, such as the transformer-based fusion, temporal convolutional fusion, and multiscale transformer fusion.

4.6.1 Training Configurations

All training runs used the same dataset partitioning strategy. This was split into 650 training, 150 validation and 208 testing trajectories. The split was created with a fixed random seed of 42 and terrain-aware stratification to make sure the three subsets have a balanced distribution of terrain. Sliding temporal windows were taken from the synced modality streams to create training samples. Each training clip was 40 timesteps at 30 Hz which is around 1.3 seconds of real movement. Consecutive clips were extracted with an 8-frame stride, which will ensure that there is a significant overlap of consecutive clips in time but also give a much larger number of training clips. Prior to training, per channel z-score normalization was performed on all feature channels. Only subsample (20%) of the training trajectories were used for the estimate of mean and variance statistics, which were kept statistically separated from the normalisation process in the validation and test data. These normalising statistics were saved along with the model checkpoints to ensure that the scaling behaviour is the same during inference. The training goal was to minimize a weighted mean squared error loss on targets of body-frame linear and angular velocities. Higher weighting was given to rotational velocity components to highlight orientation stability during locomotion:

$$w = [w_{(v_x)}, w_{(v_y)}, w_{(v_z)}, w_{(\omega_x)}, w_{(\omega_y)}, w_{(\omega_z)}] = [1, 1, 1, 5, 5, 5].$$

The first three weights were associated with the translational velocity components, and the other three were associated with the angular velocity components. Simulator spikes were also clipped to ± 10 rad/s to suppress rare spikes in angular velocity targets. AdamW optimizer [68] was used to optimize all models with a learning rate of

1×10^{-4} and weight decay of 1×10^{-4} . A ReduceLROnPlateau scheduler with the reduction factor of 0.5 and the patience of 3 epochs was used to control learning rate adaptation. Training used a batch size of 128 together with early stopping using a patience threshold of 10 epochs. Modality dropout [69] was used in the training to increase the robustness of the model to sensor corruption and missing modalities. Each modality independently received a 10% probability of being masked for a given training sample. To prevent all modalities from being dropped in the same clip, a safeguard was added. This approach made sure that the fusion network did not over-rely on any particular source of sensing. Calibration `load_calibrated_joints()` was used to load the training data, which was done automatically after the front-right calf calibration correction performed during preprocessing. This made sure that the geometry of all the kinematic inputs was consistent throughout the data set. Table 3 lists the training configurations.

Table 3: Training Configurations

Knob	fusion_v1	fusion_v1_marg	fusion_v2	fusion_v2_marg	fusion_tc	fusion_mvit
Train / Val / Test	650/150/208	650/150/208	650/150/208	650/150/208	650/150/208	650/150/208
Split strategy	Stratified by terrain, seed=42	same	same	same	same	same
Clip length	40 frames	same	same	same	same	same
Clip stride	8	same	same	same	same	same

Normalizat ion	Per-channel z- score, 20% train subsample	same	same	same	same	same
Optimizer	AdamW	same	same	same	same	same
Learning rate	1e-4	same	same	same	same	same
Weight decay	1e-4	same	same	same	same	same
LR scheduler	ReduceLROnPla teau, factor 0.5, patience 3	same	same	same	same	same
Early stopping patience	10 epochs	same	same	same	same	same
Max epochs	50	same	same	same	same	same
Batch size	128	same	same	same	same	same
Modality dropout	10% per modality, drop- all safeguard	same	same	same	same	same
Angular clipping	± 10 rad/s	same	same	same	same	same
Loss	Weighted MSE [1,1,1,5,5,5]	same	same	same	same	same
Label frame	Body-frame, all axes	same	same	same	same	same

Hardware	RTX 5060 Ti 16 GB	same	same	same	same	same
Training wall-clock	~37 min	~38 min	~82 min	~60 min	~88 min	~172 min
Best epoch	7	11	31	18	29	34

4.7 Fusion Transformer Architecture

The core architecture developed for this work is FusionTransformer. It operates as a factorized modal-then-temporal causal transformer [69] called FusionTransformer. The system processes each modality stream separately, initially, in a first embedding process with synchronized temporal clips from the preprocessing phase. After this, the model performs multimodal temporal reasoning in an alternating manner between modality-based and temporal-based attention. The reason for choosing this factorized design is to minimize the computational complexity and also to allow for a lot of interaction between the various sensor inputs with an efficient architecture.

4.8 Intermediate Fusion Models.

4.8.1 Fusion_v1.

The first FusionTransformer configuration was conducted by combining only two modalities. They are joint kinematics features with 31 dimensions and IMU only inertial features with 10 dimensions. This configuration was established before the completion of the visual odometry preprocessing pipeline and was the first multimodal fusion stage of the study. This stage was referred to as fusion_v1. Input tensor followed the below structure where $\mathbf{B}$ = batch size and $\mathbf{T}$ = 40 represents the temporal sequence length.

$$kin \in R^{(B \times 40 \times 31)}, ins \in R^{(B \times 40 \times 10)}$$

These tokens are then stacked in a tensor and further augmented by a learned modality embedding vector. This is an essential operation so that the network can identify the specific

modalities when involved in cross-modal attention operations. This is essential, as it allows the network to know the modality of the selected attentional field during cross-modal attention operation. Also, sinusoidal positional encodings [70] are introduced along the temporal axis to ensure the model respects the sequential order of the data.

4.8.1.1 Factorized Model-Temporal Attention.

The transformer backbone consists of two duplicated factorized attention blocks, breaking up the processing into two stages. First, modal attention applied over tokens at individual timestep. After that temporal attention is applied across the timesteps of each modality stream. This hierarchy allows the network to share information among these sensing sources first in order to start reasoning how this motion changes over time. Maintaining compatibility with real-time streaming, a causal upper-triangular masking [71] strategy is used for temporal attention. This means that none of the future data could leak into the earlier representations, thereby making sure that any prediction at timestep t will be based on current and past observations, but not anything in the future. The fusion_v1 architecture has 437,382 trainable parameters, with each block set with 4 attention heads and a fusion Feed-Forward layer with 256 dimensions. And dropout rate was 0.1.

4.8.1.2 Output Prediction Head of Fusion_v1

Lastly, an average of the modality outputs (after the last transformer block) is used to generate a uniform temporal representation. Next model takes out representation from the last timestep and sends it through the Layer Normalization [71] layers. Finally, the 128 dimensional feature is transformed to six regression outputs by the linear projection layer, from which the body-frame translational and rotational velocities are estimated for motion estimation.

4.8.2 Fusion_v1_MARG.

In addition to the fusion_v1, the research explored another inertial configuration that focuses on magnetometer-augmented fusion. This is a special version of the model, called fusion_v1_marg, that has been designed to investigate the effect of adding a heading reference to motion estimation using only joint kinematics. While the fusion_v1 relies on IMU-only data,

the MARG versions utilize specialized preprocessing outputs that integrate accelerometer and gyroscope readings with magnetometer measurements. This configuration uses two pass tilt compensated yaw estimation procedure to assure heading stability. Magnetometer data from the simulator is in the "North-East-Down" convention, which is transformed to "East-North-Up", the convention used by the filter, as follows: $(m_y, m_x, -m_z)$. Training conditions for this variant were kept very similar to the fusion_v1. The same transformer, loss function and optimization parameters were used, and the only difference between them was the preprocessing source of the inertial data.

4.8.3 FusionTransformers with Visual Odometry – Main Model

After obtaining the visual motion estimates for all the trajectories in the visual odometry preprocessing stage, the transformer network was extended to incorporate the visual motion estimates. This new configuration, called fusion_v2, added a third modality stream which was based on DROID-SLAM [66] motion features extracted in 7 dimensions. This visual feature vector is the per-frame SE(3) displacement vector [67] from previous camera frame represented as $[\Delta t_x, \Delta t_y, \Delta t_z, \Delta q_x, \Delta q_y, \Delta q_z, \Delta q_w]$. This version combined all the multimodal input information, including kinematics, IMU-only inertial data and visual odometry features. For visual odometry the same two layer MLP embedding approach was used for the other modalities, with a 7 dimensional vector being embedded into the common 128 dimensional latent space. One of the major benefits of this design is that little or no structural changes were needed to integrate the additional sensing source into the transformer design. Moving from two modalities to three only necessitated the addition of one modality embedding vector and a single modality-specific MLP encoder. The rest of the core components (temporal attention block, modal attention block, positional encoding, prediction head) stayed exactly the same as in the first version. The architecture finally became fusion_v2 with a total of 455,046 trainable parameters.

In addition to the standard version, a magnetometer-augmented variant designated fusion_v2_marg was investigated. This magnetometer-augmented variant, used all three modalities of the sensors, kinematics, visual odometry and magnetometer-augmented inertial fusion, in a unified representation. In particular, it is an extension of a three-modality

transformer, in which the IMU-only stream is replaced with the stream of inertial outputs produced by the MARG. The architecture is the same as the previous version with the exception of the type of inertial preprocessing source (as described in `fusion_v1_marg`). The factorized modal-temporal transformer structure was kept the same as it is used in the standard transformer. The positional encoding strategy was also identical to that of the standard transformer. The prediction head and optimization settings were also not changed. The total number of trainable parameters is the same as in the original model (455,046), the model only uses different input representation, so it does not change the structure of the model. This holistic framework enables the transformer backbone to learn from various data feeds, including high-frequency proprioceptive data, a stable heading reference, and accurate frame-to-frame visual motion estimation, to enhance performance. Figure 2 shows the FusionTranformer model pipelines.

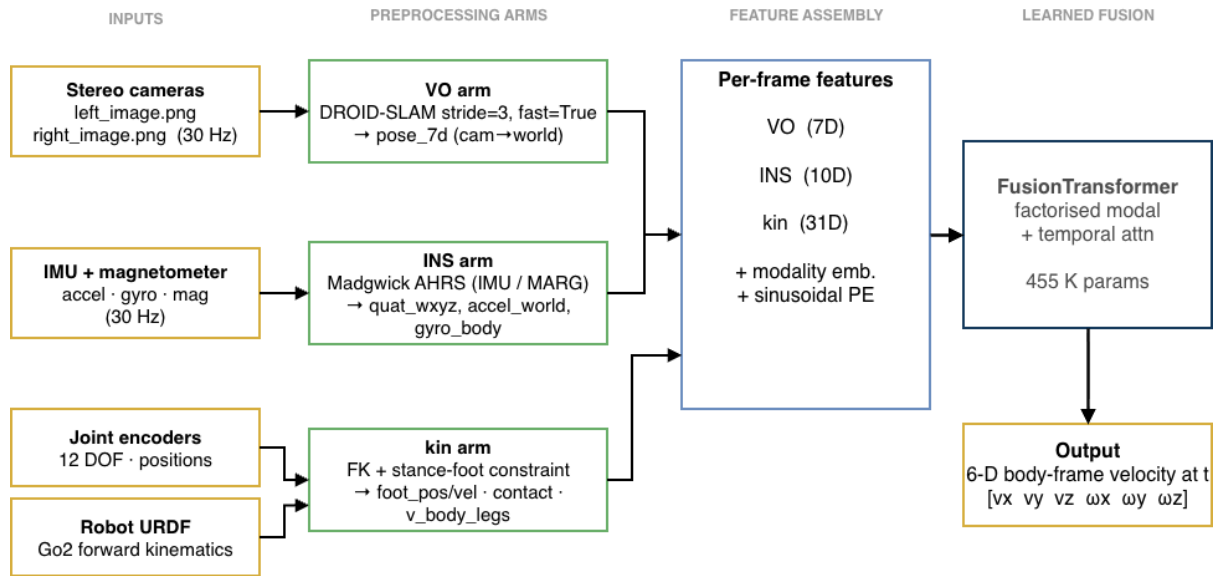


Figure 2: Fusion transformer pipeline

4.8.4 Fusion TCN Architecture – Alternative Model

A temporal convolutional alternative model, FusionTCN, was created, to check whether explicit attention mechanisms were necessary for multimodal temporal fusion. This architecture is based on R3D-18 residual video networks, but has been modified for use with one-dimensional sensor data. Because these sensor streams lack spatial dimensions, the original three-dimensional convolutions were reformulated as one-dimensional temporal convolutions. In this approach, modalities are merged in the post embedding phase in the feature channel by concatenating. Firstly, each modality of the senses is fed into a separate linear projection layer followed by a GELU layer at each timestep. In particular, the 31 dimensional kinematics stream is projected to 32 dimensions, the 10D inertial stream is projected to 16D and the 7D visual odometry stream is also projected to 16D. These are then concatenated together to form one temporal 64 channel tensor for the convolutional stages. The base structure of the network is a temporal stem convolution and four residual stages with two BasicBlock1D layers in each stage. As data moves through the stack, the architecture increases channel dimensionality moving from 64 to 256. And also simultaneously reduced temporal resolution via stride-2 downsampling between stages. The temporal resolution follows a decreasing sequence from 40 down to 5 timesteps. The backbone is based on Residual Skip Connections [62] to guarantee the stability of the optimization and a healthy gradient flow through this hierarchy. The output of the last convolutional stage is then flattened by taking the average over the time dimension. This pooled representation is then fed to a linear projection layer that outputs the 6 regression outputs: body-frame linear and angular velocity. Overall, the FusionTCN architecture comprises 1,023,798 trainable parameters. Figure 3 shows the FusionTCN diagram.

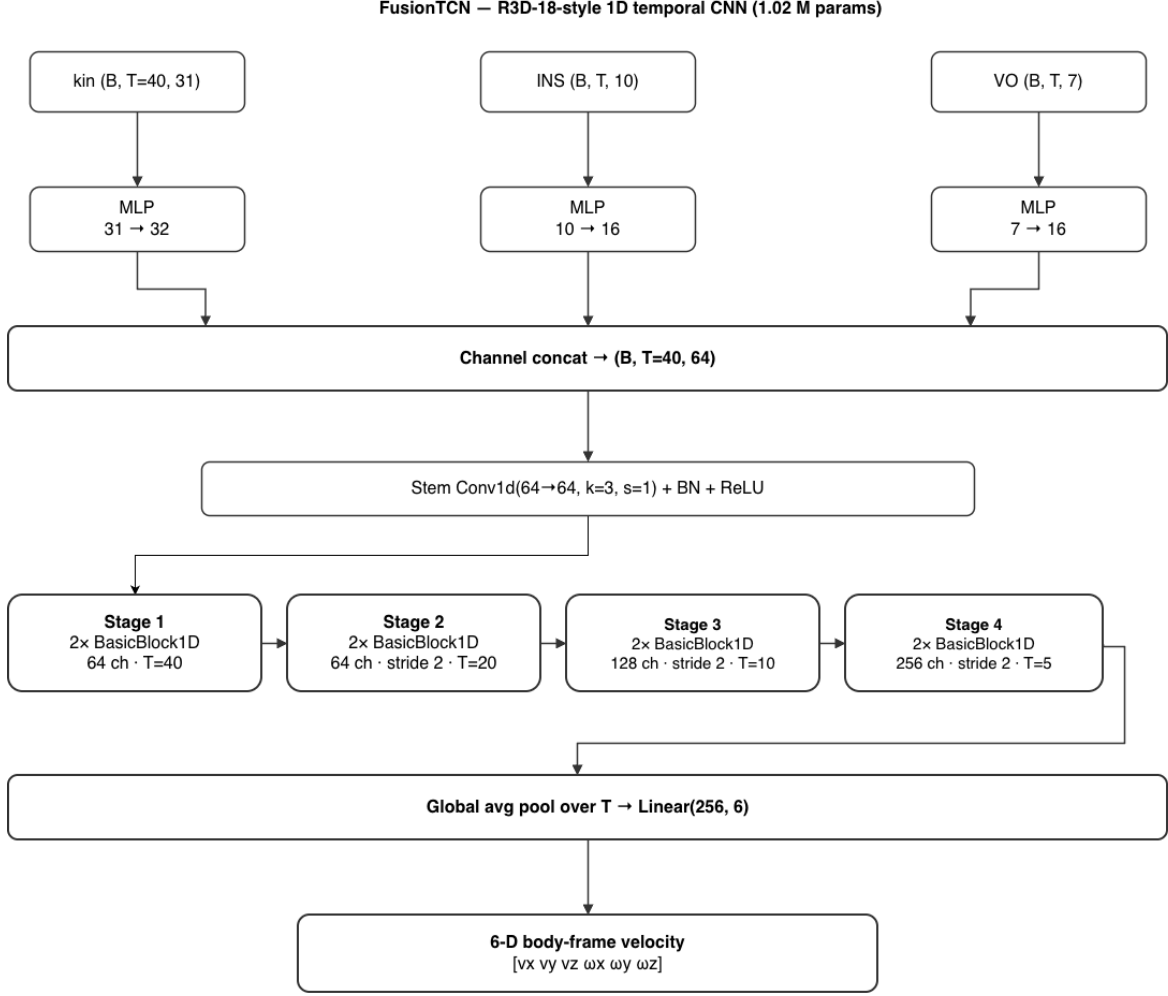


Figure 3: Fusion TCN architecture

4.8.5 FusionMViT Architecture- Alternative Model

Apart from the convolutional approach, a multiscale transformer architecture called FusionMViT was also introduced as another alternative model to explore the fusion with hierarchy framework by adhering to temporal aspect. This design builds on the basic ideas of Multiscale Vision Transformers (MViT) [60] to handle sequential fusion of input from various sensors. Instead of the traditional MViT models which process the image patches, this architecture works with multimodal temporal tokens of shape (B, T, M, D) , including the modality count, M . The architecture comprises of 3 progressive transformer stages. This is

done with average pooling, resulting in a loss of temporal resolution, and an increase of dimensionality in the channel (feature) space. In particular, the temporal length is decreased by half at each step of the stage size increase: 64 to 96 to 128, while the temporal length is shortened: 40 timesteps to 20 to 10. All of these stages have 2 factorised modal/temporal attention blocks, 4 attention heads and feed-forward expansion layers with a factor of 2. A crucial wrinkle in this alternative, relative to the causal FusionTransformer is that FusionMViT does not require causal temporal masking. This is because the model is designed to produce a single regression prediction from the entire temporal clip rather than streaming predictions. Temporal reduction is introduced on the model by employing average pooling on neighbouring timesteps. This method is more straightforward in terms of implementing the attention models across different scales in the multiscale structure while there are still hierarchical abstraction methods shared by MViT architectures. After arriving in the last transformer stage, the network averages the features in BOTH modality and temporal space. This is fed sequentially into Layer Normalization [71] and another linear regression layer to produce motion estimates. Overall, FusionMViT's network contains 758,566 trainable parameters. Figure 4 shows the architecture diagram of the FusionMViT.

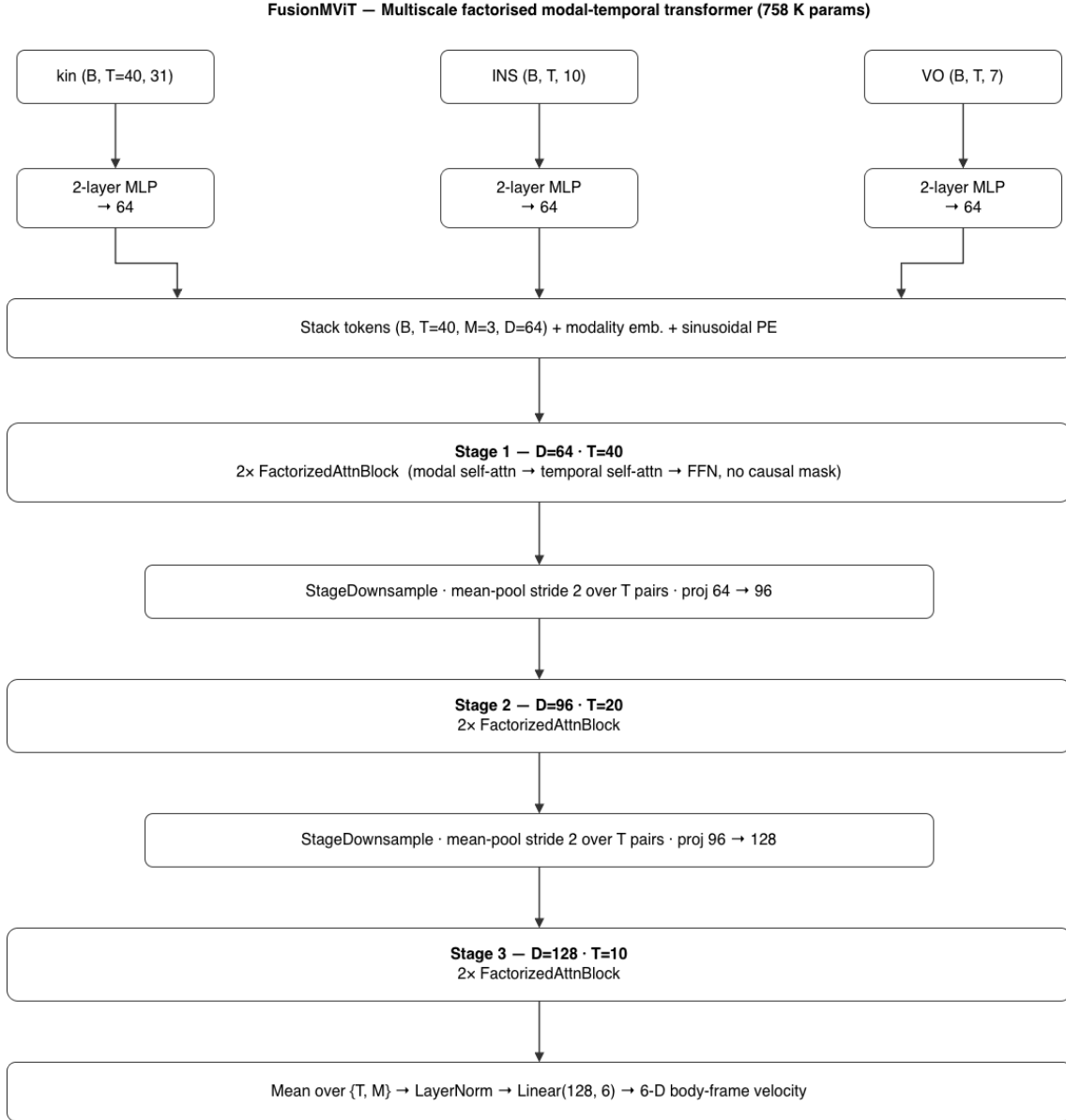


Figure 4: Fusion MViT diagram

4.9 Testing on Real World Data

A real world dataset, collected by Mechadroits Ltd, became available to test with at the very end of the research. This dataset was intended for future research. However, the timing of the availability of the dataset allowed for testing of some of the models trained here on the new dataset in order to get some initial benchmark results. The dataset was collected on a Unitree

GO2, with a Stereolabs Zed 2i stereo camera, a Holybro Pixhawk 6X providing IMU data, as well as a Holybro M9N module providing GPS and compass. The robot was manually controlled for 1702 seconds, collecting 101,960 sets of data from all sensors, performing 16 loops, where each loop was started at a stationary aruco marker board, which was used for calculating the ground truth position. When the aruco marker board was not visible in the Zed camera, dead reckoning based on the state estimation was performed, with errors averaged over the span until the aruco marker board was visible again. The data collection was performed Murphy's Bush, which is New Zealand native bush, in South Auckland (Figure 5).



Figure 5: The robot and aruco marker board used for data collection.

5.0 Results and Discussion

5.1 Initial Pilot Experiments with End-to-End Visual Models

The dataset was collected in a simulated forest setting in Isaac Sim, which was designed to resemble natural conditions. The robot moved through a densely populated scene with approximately 150 trees that were placed in a naturalistic manner to reduce the repetitive structure ambiguity. The navigation involved various types of terrains, including flat, uneven, and steep slopes. Domain randomization was applied across both visual and physical parameters.

The dataset contained 12 trajectories and approximately 43,200 frames. These were split into training, validation, and test sets at the trajectory level to avoid temporal overlap between splits.

In this study, four model configurations were examined.

- R3D-18 3D CNN using RGB input
- R3D-18 3D CNN using RAFT-Stereo Disparity
- MViT using RGB
- MViT using RAFT-Stereo Disparity

The performance of all the models was evaluated using root mean square error (RMSE) on normalized 6-DOF velocity predictions. The evaluation further distinguished between linear velocity error (m/s) and angular velocity error (r/s).

5.1.2 Validation Results of Pilot Dataset

The validation set was employed to choose the most appropriate checkpoint of each model. The model that had the lowest validation loss was selected to be evaluated. The lowest validation RMSE of 0.1910 was found in the MViT model with RAFT disparity across all the models. The R3D-18 CNN with RGB input followed with an RMSE of 0.2059. The other models presented slightly higher errors.

These results show that the MViT model benefits from the disparity input and converges quicker when depth input is present, and the CNN model is good when it is fed RGB inputs that carry rich texture and colour data. However, this stage only guides model selection. The test results (Table 4) draw the final conclusions.

Table 4: Validation Results of Pilot Dataset

Model	Visual Input	Best Epoch	Val Loss	Val RMSE	Lin RMSE (m/s)	Ang RMSE (rad/s)
R3D-18 CNN	RGB	36	0.0424	0.2059	0.1478	0.2509
R3D-18 CNN	RAFT Disparity	21	0.0474	0.2176	0.1607	0.2625
MViT	RGB	45	0.0485	0.2202	0.1157	0.2891
MViT	RAFT Disparity	29	0.0365	0.1910	0.1165	0.2436

5.1.3 Test Set Results of Pilot Dataset

The test set contains 3 trajectories that were not seen during the training or validation phase. Final evaluation was done after using the best checkpoint that was selected on the validation set. The complete test results are presented in the Table 5.

Table 5: Test Results of Pilot Dataset

Model	Visual Input	Test Clips	Test Loss	Test RMSE	Lin RMSE (m/s)	Ang RMSE (rad/s)
R3D-18 CNN	RGB	2493	0.0542	0.2328	0.1688	0.2827
R3D-18 CNN	RAFT Disparity	2493	0.0599	0.2448	0.1692	0.3021
MViT	RGB	1244	0.0662	0.2572	0.1226	0.3425
MViT	RAFT Disparity	1244	0.0511	0.2260	0.1377	0.2885

The architectural changes in terms of the number of clips (2493 in CNN and 1244 in MViT) correspond to the difference in the design of the temporal window: CNN has $T=8$ frames with $\text{stride} = 4$, while MViT uses $T=16$ frames with $\text{stride}=8$. MViT + RAFT Disparity has the best overall test loss and RMSE metrics. R3D-18 CNN + RGB ranks second in terms of overall RMSE (0.2328), and it is ahead of both R3D-18 CNN + RAFT (0.2448) and MViT + RGB (0.2572). The overall RMSE ranking is preserved from validation to Test. But the component ordering shifts on angular RMSE, R3D-18 CNN + RGB (0.2827) overtakes MViT+RAFT (0.2885) on the test set, having ranked second in validation. This divergence motivates the per-component analysis in Section 5.4.

5.1.4 Architecture Comparison: MViT vs R3D-18 CNN

Table 6 isolates the effect of architecture by comparing MViT and R3D-18 CNN on matching visual inputs.

Table 6: Architecture Comparison on Test Dataset

Visual Input	CNN Test RMSE	MViT Test RMSE	Difference
RAFT Disparity	0.2448	0.2260	MViT better by 7.7%
RGB	0.2328	0.2572	CNN better by 9.5%

With RAFT disparity, MViT performs 7.7% better than R3D-CNN. The multi-head self-attention mechanism of the transformer takes care of spatial relationships across the entire image, making it ideal for exploiting the metric depth structure recorded in disparity maps [67]. The depth cues are distributed across the entire visual field, and the global attention provides an inter-region depth relationships, which local convolutions perform only within their receptive fields.

With the RGB input, the reverse happens: CNN performs better than MViT with 9.5% difference. The visual density of the forest environment and tree canopies, the foliage, and the ground cover generate the high-frequency texture at all scales. R3D-18's 3D convolutions effectively extracts local motions and texture gradients from these patterns. MViT's global

attention spreads representation across the entire frame, potentially diluting the strong local texture signals that are the major distinguishing feature in RGB forest imagery.

It is one of the main findings of the interaction between input modality and architecture. The two architectures do not always outshine the other; the best architectural decision will be based on the visual input that one is working with.

5.1.5 Input Modality Comparison: RGB vs RAFT-Stereo Disparity

Table 7 compares visual modalities within each architecture.

Table 7: Input Modality Comparison on Pilot Training

Architecture	RGB Test RMSE	RAFT Test RMSE	Better Modality
R3D-18 CNN	0.2328	0.2448	RGB by 4.9%
MViT	0.2572	0.2260	RAFT by 12.1%

In the case of MViT, RAFT disparity outperforms RGB test RMSE by 12.1%. Disparity maps encode metric depth directly, giving an absolute geometric reference that is not available in RGB images. The global attention of the transformer incorporates such depth relationships throughout the entire spatial representation in every frame, which translates to improved velocity estimation.

For R3D-18 CNN, RGB is 4.9% better than RAFT disparity. The 3D convolutional filters of CNN are optimized to extract local colour and texture patterns. When input is switched to RAFT disparity, the pre-computing process produces a single grayscale image and replicates the single channel to three channels to match the CNN input format. This removes colour and texture cues the convolutional filters used in the local feature extraction and decreases the overall performance on the texture-rich forest terrain.

5.1.6 Validation to Test Generalization

Table 8 shows the degradation from validation to test RMSE for each model configurations.

Table 8: Validation to Test Generalization

Model	Visual Input	Val RMSE	Test RMSE	Degradation
R3D-18 CNN	RGB	0.2059	0.2328	+13.1%
R3D-18 CNN	RAFT Disparity	0.2176	0.2448	+12.5%
MViT	RGB	0.2202	0.2572	+16.8%
MViT	RAFT Disparity	0.1910	0.2260	+18.3%

All the four models degrade by 12-18% from validation to test, which is expected as the test trajectories have different walking paths and scene configurations that were not seen during training. The R3D-18 CNN models exhibit tighter generalization (12.5 - 13.1%), whereas MViT models have slightly higher degradation (16.8 - 18.3%). The degradation is moderate and consistent across all configurations, which means that the training is stable and not overfitting severely to the training set.

5.2 Scaling to the Full Dataset

5.2.1 R3D-18 CNN RGB : Baseline Visual Performance

The first visual model tested, `cnn_rgb`, was trained end-to-end on rgb video clips with 8 frames of 30 Hz. It was evaluated on the held out validation split (150 trajectories) with an overall RMSE of 0.5173. The breaking down of this error gave a linear RMSE of 0.1601 m/s and angular RMSE of 0.7138 rad/s. These performance measures are higher than those achieved on the small-scale forest pilot dataset. This indicates that the end-to-end model did not scale well to the full dataset.

Even though the model receives RGB frames and raw inputs from the IMU, magnetometer, and joint state sensors, these sensor streams were not preprocessed before being passed to the model. There was no physics-based orientation filter that turns the raw IMU measurements into a stable estimate of the body orientation, no forward kinematics computation that turns joint angles into a velocity estimate in the body-frame, and no explicit normalization of the visual motion signal. Consequently, the network must be trained to perform IMU integration, forward kinematics, visual ego-motion estimation and multimodal fusion all in one in parallel using gradient descent. This makes it a significantly more challenging optimization problem than the

second fusion approach described later that is based on intermediate representations that are generated prior to temporal fusion and that do not change from one fusion to another. The experiments conducted at large scales on RGB data suggest that the network had difficulties to directly learn the intermediate physical relationships from raw streams and image sequences.

A staged approach was therefore introduced. To avoid forcing the visual network to directly predict body-frame velocity from raw multimodal inputs, the architecture was changed to a visual branch, which first learned some intermediate types of visual feature representations. These embeddings were then fused with sensor embeddings by a fusion head which is trained with light backbone for the final velocity estimation task.

5.2.2 Stage-2 R3D-18 CNN RGB : Test Set Evaluation

The Stage-2 R3D-18 CNN RGB configuration was tested on the full 208-trajectory test set containing 93,767 temporal clips. Table 9 shows the aggregate estimation metrics and root mean squared errors for each axis.

Table 9: Stage-2 3D CNN RGB Test Set Performance

Axis	RMSE
v_x (m/s)	0.1020
v_y (m/s)	0.1031
v_z (m/s)	0.0537
ω_x (rad/s)	0.1887
ω_y (rad/s)	0.1171
ω_z (rad/s)	0.0928
Linear RMSE	0.0893 m/s
Angular RMSE	0.1389 rad/s
Overall RMSE	0.1168

Even though the network was trained for about 120 hours using a model with 33.5 million parameters, the network had a significant residual error for each of the six motion axes. The per-axis error distribution gives insights on the limits of the end-to-end visual learning approach.

The forward motion was thought to be the simplest to estimate as this motion results in strong optic expansion in the image plane. The errors for forward velocity (0.1020 m/s) and lateral velocity (0.1031 m/s) were almost the same, however. This indicates that the model had learned something about the overall intensity of the locomotion rather than the motion across each body-frame axis.

Vertical velocity had the lowest linear error of 0.0537 m/s, which may be due to the fact that vertical motion of the body is related to body pitch and camera bounce that causes more distinct visual variations in the image plane.

Regarding the angular components, the yaw rotation was the most successful with a value of 0.0928 rad/s as the yaw rotation creates strong movement of the scene horizontally and is visually different. Roll rotation, on the other hand, resulted in the largest error (0.1887 rad/s). This is because during locomotion, the roll oscillations only cause small vertical image displacements, which are hard to detect from terrain variation and camera vibration.

These results suggest that the observability of each of the motion components from the forward-facing camera perspective was a significant factor on the residual error. In general, the experiments demonstrated that even an end-to-end visual learning with two stage training and the raw sensor inputs were not enough to provide reliable estimation of the six axis body frame velocity. This encouraged moving to the multimodal fusion framework based on preprocessing.

5.2.3 MViT RGB : Validation Performance

As per the initial direction of this study, multiscale video transformer model, MViT RGB, was trained, expecting it would outperform CNN-based architecture. To train the model, the same

data set and the same sensor configuration were used and the model was tested with the validation set which includes 150 trajectories. It achieved an overall RMSE of 0.3977, a linear RMSE of 0.0748 m/s, and an angular RMSE of 0.5574 rad/s. MViT RGB v3 outperformed CNN RGB v2 on the same split of the validation set by 23% on the overall RMSE and 53% on the linear RMSE.

The gain provides an indication of the multi-scale representational ability of the transformer to model spatial and temporal aspects, and creates a more detailed representation of motion across the clip. This improvement has been made but high angular errors have remained. Even though there are a lot of multi-head attention models for longer temporal contexts, the estimation of angular velocity of the body frame from vision alone is intrinsically limited by the angular RMSE of 0.5574 rad/s. A stereo camera forward facing does yield optic flow when the camera is rotated, but in the absence of range data is unable to differentiate rotational from translational flow and sideways or up/down velocity when camera axis is parallel to direction of motion gives very weak flow signals. Again this is not a limitation of the ability of the model, but inherent to mono geometry. This promotes the use of multimodal fusion method based on structured inertial and kinematic pre-processing.

Additionally, the model with transformer was quite expensive in terms of computational demands when compared with the CNN based approach. The training was much slower (17000 – 42000 s per epoch), required a lot of GPU memory and the results of angular estimation were not yet good enough to deploy. So this model was not used for the final test-set evaluation and deployment integration experiments.

A similar trend was observed for the experiments conducted with CNN and MViT. Although useful for general locomotion behaviour and forward velocity estimation, raw RGB video did not provide reliable information to accurately estimate the six axis body-frame velocity. These results were the motivation to switch to the multimodal fusion framework depending on the preprocessing, which is presented in the rest of this chapter.

5.3 Fusion Models

All fusion models were evaluated on the identical 208-trajectory test set. This contains 92,935 clips, enabling direct comparison across all fusion configurations.

5.3.1 Fusion v1: Two-Modality Transformer Baseline

The first fusion experiment (fusion_v1) used a factorized modal and temporal attention. This setup fusing 31D kinematics and 10D INS data, without the use of visual odometry. The architecture was found to be efficient in terms of computation and only needed 437,382 parameters. This meant that the training took place in 37 minutes. Performance data for this version is shown in detail in Table 10 .

Table 10:Fusion_v1 Test Set Performance

Axis	RMSE
v_x (m/s)	0.0786
v_y (m/s)	0.0554
v_z (m/s)	0.0540
ω_x (rad/s)	0.1616
ω_y (rad/s)	0.0863
ω_z (rad/s)	0.0806
Linear RMSE	0.0637 m/s
Angular RMSE	0.1156 rad/s
Overall RMSE	0.0933

This is a significant improvement from the Stage-2 CNN RGB. Overall RMSE fell by 20.1% from 0.1168 to 0.0933. And also angular RMSE improved by 16.8% from 0.1389 to 0.1156 rad/s. The linear RMSE showed a 28.7% improvement, dropping from 0.0893 to 0.0637 m/s.

The use of the multi-modality has evidently been effective in obtaining complementary information which cannot be obtained by the visual-only pipeline. The key observation at this step is the ‘per axis pattern’. Vertical velocity v_z has the lowest linear error at 0.0540 m/s, with lateral velocity v_y is close behind at 0.0554 m/s. The worst forward velocity v_x is at 0.0786m/s. At first glance this is a surprising finding, because one might expect that, if the stance-foot velocity is kinematic derived, it would do a good job of controlling body translation also, because leg contacts directly control body translation. The reason is that the finite-difference noise in the kinematic computation is at 30 Hz. The stance-foot velocity estimate is based on centred difference over foot positions two frames apart (67 ms), which will lead to large short duration spikes in the differentiated velocity estimate corresponding to rapid leg-contact events (foot strike and/or lift). This is most noticeable on the forward axis as the Go2 gait is mostly anterior-posterior foot movement and the noise is going to increase the error distribution for v_x at the tail.

The most difficult axis is still the roll ω_x with a value of 0.1616 rad/s which is the largest error of any axis. The roll axis presents the greatest challenge. In the Go2 trot, the body's oscillation at 2 Hz is $\pm 5^\circ$, causing high frequency gyroscope spikes. The INS arm is able to pick up these spikes, but they are not present in the kinematic features. In addition, without a drift-free orientation reference the IMU-only INS arm cannot fully resolve these transients, allowing them to propagate into the model's roll-rate predictions. This restriction leads to an overestimated distribution at the tail of the ω_x distribution with the P95 value of 0.2704 rad/s while the median is much lower at 0.0767 rad/s. This is not observed in the median, but rather in the P95 tail, and therefore only very few frames with a numerically unstable kinematic derivative have a negative impact on the overall RMSE.

5.3.2 Fusion v1 MARG: Adding Magnetometer Data

The fusion_v1_marg version is an upgrade on top of the previous one, fusion_v1, with an improved INS arm. The 6 axis IMU-only setup was changed to a MARG (Magnetic, Angular Rate, and Gravity) setup which allows the Madgwick AHRS [73] filter to use an absolute heading of yaw. Even with this architectural change, the model was still very efficient, retaining

the same number of parameters (437,382) and the same training time (38 minutes). The performance improvements from this are listed in Table 11.

Table 11: Fusion_v1 MARG Test Set Performance

Axis	RMSE
v_x (m/s)	0.0724
v_y (m/s)	0.0504
v_z (m/s)	0.0502
ω_x (rad/s)	0.1391
ω_y (rad/s)	0.0875
ω_z (rad/s)	0.0734
Linear RMSE	0.0586 m/s
Angular RMSE	0.1039 rad/s
Overall RMSE	0.0844

All metrics improved compared to fusion_v1. Overall RMSE dropped from 0.0933 to 0.0844, a 9.5% improvement. Angular RMSE fell from 0.1156 to 0.1039 rad/s, a 10.1% improvement. Linear RMSE improved from 0.0637 to 0.0586 m/s, an 8.0% reduction. Yaw axis ω_z clearly improved per axis from 0.0806 to 0.0734 rad/s with the main reason for improvement being the absolute yaw reference added in the MARG configuration.

The improvement on ω_x (roll) was also notable: 0.1391 rad/s versus 0.1616 rad/s, a 13.9% reduction. This is a bit of a surprise, as it is the magnetometer that is mainly used to limit yaw. The likely reason is indirect. By providing a drift-free yaw, the model maintains a steady quaternion reference throughout the 1.3 second window. This allows it to evaluate sudden, transient gyro spikes against a consistent body orientation, minimizing the leakage of yaw error to the roll estimation. Because the gyro signal is measured in the body frame and remains identical in both setups. The performance boost comes from entirely from having a more steady orientation context, not from any changes to the raw angular rates.

A meaningful milestone was reached when two linear axes, v_y and v_z , fell to their lowest values so far, at 0.0504 and 0.0502 m/s respectively.

The most difficult linear axis was forward velocity v_x with 0.0724 m/s. Even with these improvements, the overall linear RMSE (0.0586 m/s) and angular RMSE (0.1039 rad/s) indicate that the INS and kinematic arms alone were not able to compensate for the rest of the error sources.

5.3.3 Fusion v2: Introducing Visual Odometry

The visual odometry arm was added in the fusion_v2 iteration, which introduces 7D SE(3) deltas from DROID-SLAM. These were added to the current kinematic and INS features at stride-3 sampling interval. The model complexity had been raised to 455,046 parameters with this additional modality, taking 82 minutes to train the model. This choice of stride-3 subsampling came from an empirical sweep over five stride values. This evaluation was performed for three sample trajectories that cover the range of terrain variations of the study (Figure 5.3). Results showed that the weighted-average Absolute Trajectory Error (w-ATE) for Stride=3 was 0.33 m for the terrain mix of 62% forest, 28% flat and 10% uneven. Stride=3 was found to be $2.7\times$ faster than stride=1, which had a w-ATE of 0.36 m, without sacrificing trajectory accuracy. A "performance cliff" was reached when the stride was increased further and the ATE increased 78% at Stride=4 on flat terrain. The performance of this configuration in detail is shown in Table 12.

Table 12: Fusion_v2 Test Set Performance

Axis	RMSE
v_x (m/s)	0.0562
v_y (m/s)	0.0425
v_z (m/s)	0.0453
ω_x (rad/s)	0.1272

ω_y (rad/s)	0.0737
ω_z (rad/s)	0.0630
Linear RMSE	0.0483 m/s
Angular RMSE	0.0924 rad/s
Overall RMSE	0.0737

Fusion_v2 is the best-performing model in this study. The addition of the visual odometry arm reduced overall RMSE from 0.0844 (fusion_v1_marg) to 0.0737, a 12.7% improvement. For the first time linear RMSE with a value of 0.0483 m/s, which is a 17.6% improvement over fusion_v1_marg. Angular RMSE dropped to 0.0924 rad/s, the first time angular RMSE also dropped below 0.10 rad/s, and a 11.1% improvement.

Fusion_v2 has 73 times fewer parameters than the Stage-2 CNN RGB v2 model (455,046 vs. 33,485,894) and trains 88 times faster (82 minutes vs. ~120 hours).

The contribution of the VO arm is most apparent on the forward velocity axis, v_x , where it decreased from 0.0724 m/s (fusion_v1_marg) to 0.0562 m/s, an improvement of 22.4%. This is in line with DROID-SLAM providing accurate forward translation estimates on the forward facing camera where the optic flow signal is strongest. Even the noisier lateral VO estimates have some useful information which the network is able to exploit, as evidenced by the improvement in v_y (from 0.0504 to 0.0425 m/s), although it is less than that on the forward axis.

The ongoing weakness of ω_x (0.1272 rad/s) is noteworthy. The VO arm does provide rotation estimates and one might think that the rotation provided by the DROID-SLAM arm could be used to reduce the uncertainty in the roll estimate. In practice, at 2Hz, the resulting camera-plane translations due to the roll oscillation are too small to be reliably matched by DROID-SLAM's point-correspondence matching. A likely explanation is that the SLAM back-end interprets vertical image motion as scene-point depth uncertainty rather than true camera rotation. This means that there is no reliable roll VO angular estimate and the network learns

to discount VO angular contributions for ω_x . The IMU gyro remains the primary roll- rate signal. The difficulty is there because the body-frame label smooths out the 2Hz gait roll oscillation that gyro itself reads faithfully.

5.3.4 Fusion v2 MARG: Magnetometer in the Full Three-Modality System

The MARG configuration was explored in the full three modality system (kinematics, INS-MARG and VO) for the fusion_v2_marg iteration. This enabled the model to use the absolute orientation of the MARG sensor, along with the translational and rotational estimates from the VO arm. With the full sensor suite, the model still had the same number of parameters: 455,046. Interestingly, the training process was more efficient than the standard fusion_v2, completing in 60 minutes. The final results for this 3 modality architecture are reported in Table 13.

Table 13: Fusion_v2_marg Test Set Performance

Axis	RMSE
v_x (m/s)	0.0636
v_y (m/s)	0.0459
v_z (m/s)	0.0486
ω_x (rad/s)	0.1425
ω_y (rad/s)	0.0808
ω_z (rad/s)	0.0677
Linear RMSE	0.0533 m/s
Angular RMSE	0.1023 rad/s
Overall RMSE	0.0816

The outcomes are remarkable and surprising: When the magnetometer was added to the previously best system, it actually reduced its accuracy. The overall RMSE degradation was from 0.0737 to 0.0816, which is a 10.7% degradation in performance. This degradation was not restricted to just one of the metrics. But both linear RMSE and angular RMSE suffered

compared to fusion_v2. The per-axis data illustrates this shift clearly: v_x increased from 0.0562 to 0.0636 m/s, while ω_x moved from 0.1272 to 0.1425 rad/s. Perhaps most surprisingly, ω_z degraded from 0.0630 to 0.0677 rad/s, that is, the precision of the yaw axis which the MARG configuration is especially designed to protect, suffered measurable degradation.

To understand why, consider what the MARG configuration adds: primarily absolute yaw reference from the magnetometer. In the two modalities (fusion_v1_marg and fusion_v1), this yaw information was very useful as another reliable yaw signal was missing in the set of features. However, in the three-modality system (fusion_v2), yaw rotation is already encoded in the Δq quaternion from DROID-SLAM. Visual frame-to-frame matching accumulates yaw information in the SLAM back-end, and as a result, a continuous yaw rate signal is obtained. A problem for the attention mechanism is created when the MARG INS arm injects a redundant but independently scaled yaw signal, in the same sequence and at the same time.

The data shows a phenomenon, which can be called the MARG paradox: A signal strictly beneficial in a two-modality system, becomes actively harmful in a three-modality system. This degradation seems to be due to redundancy of representation with the visual odometry (VO) arm.

The magnetometer and VO signals probably provide conflicting supervision during training as opposed to reinforcing the yaw estimate. The model struggles to consistently discount the noisier magnetometer signal because it is not readily apparent which one is more reliable for a specific clip. This battle effectively diluted the high accuracy VO yaw signal with the constant declination offset and noise of the magnetometer.

The first conclusion to be drawn from this result is that in fusion systems, one has to pay attention to the redundancy of signals, not just to the diversity of signals. This result shows that there are no monotonically increasing benefits of adding sensors. If two modalities contain very similar information, but distinct systematic errors, the network may eventually behave worse than with one single and clean modality.

5.3.5 Fusion TCN: Temporal Convolutional Fusion

fusion_tcn replaced the attention-based fusion mechanism with a temporal convolutional approach, using 1,023,798 parameters and 88 minutes of training. Table 14 shows the results.

Table 14: Fusion_tcn Test Set Performance

Axis	RMSE
v_x (m/s)	0.0462
v_y (m/s)	0.0372
v_z (m/s)	0.0447
ω_x (rad/s)	0.1285
ω_y (rad/s)	0.0826
ω_z (rad/s)	0.0676
Linear RMSE	0.0429 m/s
Angular RMSE	0.0964 rad/s
Overall RMSE	0.0746

The linear RMSE of Fusion_tcn is 0.0429 m/s and the angular RMSE is 0.0964 rad/s. This configuration achieves the second best performance in the study with an overall RMSE of 0.0746, only behind the 0.0737 achieved by fusion_v2. Although the overall scores are similar, the per-axis profile differs between the two models. On the linear axes, fusion_tcn outperforms fusion_v2 across the board: v_x improves by 17.8% (0.0462 vs. 0.0562 m/s), v_y by 12.5% (0.0372 vs. 0.0425 m/s), and v_z by 1.3% (0.0447 vs. 0.0453 m/s). Fusion_v2, on the other hand, outperforms across all the angular axes, including total angular RMSE (0.0924 vs 0.0964 rad/s). This difference is due to the structure that the two models use to handle temporal data. The hierarchical stride-2 downsampling applied by the TCN results in a fixed receptive field, which is suitable for the gait cycle in this case of 2 Hz. An autocorrelation structure will have a predictable regularity at about 15-frame lags at a 30 Hz sampling rate, due to the gait oscillation. The TCN's strided filters capture this periodicity particularly effectively, which

accounts for its strong linear velocity estimation. By contrast, the fixed receptive field of the TCN is less effective at detecting the irregular angular perturbations due to abrupt transitions in terrain. These events occur at different time intervals and are therefore more suitable for the attention mechanism in fusion_v2 that can selectively weight frames with high angular rate information that do not necessarily occur during the same time interval. Overall, the hierarchical temporal filtering provided by the TCN is more useful for translational estimation, while the flexible cross-modal attention of the transformer-based architecture is more useful for rotational accuracy.

The $2.25\times$ parameter count of the fusion_tcn compared to the fusion_v2 (1,023,798 vs. 455,046) and the close performance overall imply that the fusion_tcn is performing in a niche space in terms of efficiency and accuracy. It does not surpass fusion_v2 in overall RMSE, but it has a number of specific strengths that make it ideal for certain applications where limiting position drift (from integrated linear velocity) is the most important concern.

5.3.6 Fusion MViT: Multiscale Transformer Fusion

The multiscale transformer architecture was used by fusion_mvit for the last iteration to solve the fusion problem. This method required a lot of computation time, with the longest being 172 minutes for a training time, which was the biggest among all the fusion models in this study. This intensive training time did not result in a high-footprint model, though, as the model retained a mid-range footprint of 758,566 parameters. The performance measures of this multiscale approach are given in Table 15.

Table 15: Fusion_mvit Test Set Performance

Axis	RMSE
v_x (m/s)	0.0592
v_y (m/s)	0.0446
v_z (m/s)	0.0450
ω_x (rad/s)	0.1325

ω_y (rad/s)	0.0753
ω_z (rad/s)	0.0707
Linear RMSE	0.0501 m/s
Angular RMSE	0.0970 rad/s
Overall RMSE	0.0772

fusion_mvit achieved linear RMSE of 0.0501 m/s, and angular RMSE of 0.0970 rad/s. It ranked as the third overall, behind fusion_v2 and fusion_tcn and is worse on each individual axis than fusion_v2 except v_z . The v_z is marginally better than fusion_v2 (0.0450 vs 0.0453 m/s). The outcome is counterintuitive and disappointing in the light of the good performance of multiscale transformers in video understanding tasks. The input difference between the domain they are normally used in and this domain lies with the nature of input. The multiscale pyramid not only resolves a true token-count issue in video tasks, where tens of thousands of spatial patch tokens must be hierarchically pooled for attention to be feasible, but it also performs well on vision tasks. In fusion, here the input to fusion stage is already a set of processed feature vectors (one for every modality, per timestep). There are merely 120 tokens in the input with $T = 40$ timesteps and $M = 3$ modalities. When the number of tokens is small, the mean-pooling between stages will lose temporal information, while the factorised attention in fusion_v2 can keep this information. The multiscale pyramid is not a compression of redundancy, but an outright removal of signal. It is reflected by the training time, with fusion_mvit taking longer (172 minutes) than fusion_tcn (88 minutes) and fusion_v2 (82 minutes) and getting worse results on all summary metrics. The breakdown along each axis reveals that one of the best pitch results for all the fusion models is on the coarsest temporal scales, with ω_y (0.0753 rad/s) being MViT's closet axis to fusion_v2. That is about 2% behind fusion_v2's best.. However, the remaining axes are consistent and not as good as fusion_v2 and the training cost is not worth the marginal improvement per axis.

5.4 Comprehensive Fusion Model Comparison

Table 16 combines all fusion model results with the Stage-2 visual baseline to facilitate comparison.

Table 16: Fusion Models Comparison

Model	Overall	Linear (m/s)	Angular (rad/s)	Params	Train
Stage-2 CNN RGB	0.1168	0.0893	0.1389	33.5M	~120 h
fusion_v1	0.0933	0.0637	0.1156	437K	37 min
fusion_v1_marg	0.0844	0.0586	0.1039	437K	38 min
fusion_v2	0.0737	0.0483	0.0924	455K	82 min
fusion_v2_marg	0.0816	0.0533	0.1023	455K	60 min
fusion_tcn	0.0746	0.0429*	0.0964	1.02M	88 min
fusion_mvit	0.0772	0.0501	0.0970	759K	172 min

The table shows a clear Pareto frontier. From the overall and angular RMSE, fusion_v2 outperforms the other models, while fusion_tcn obtains the lowest linear RMSE value of 0.0429 m/s. But angular RMSE of fusion_tcn is slightly high. Fusion_mvit places third on both performance levels, but fails to score any individual axis (Figure 8). In addition, fusion_v2_marg is clearly dominated by fusion_v2. It confirms the MARG paradox described in section 5.3.4 Fusion v2 MARG: Magnetometer in the Full Three-Modality System. The parameter efficiency across these models is particularly striking (Figure 9). The visual model used in Stage-2 has 33.5 million parameters while all of the fusion models have less than 1.1 million parameters. This 73 $\times$ reduction of parameters from Stage-2 to fusion_v2 is accompanied by 88 $\times$ faster training (Figure 7), and a 36.9% decrease in overall RMSE.

This represents one of the clearest findings of this work. The structured preprocessing arms that compress raw sensor data into semantically meaningful intermediate representations increase the model's efficiency more than end-to-end visual learning. Figure 6 shows the overall RMSE of all models on test set.

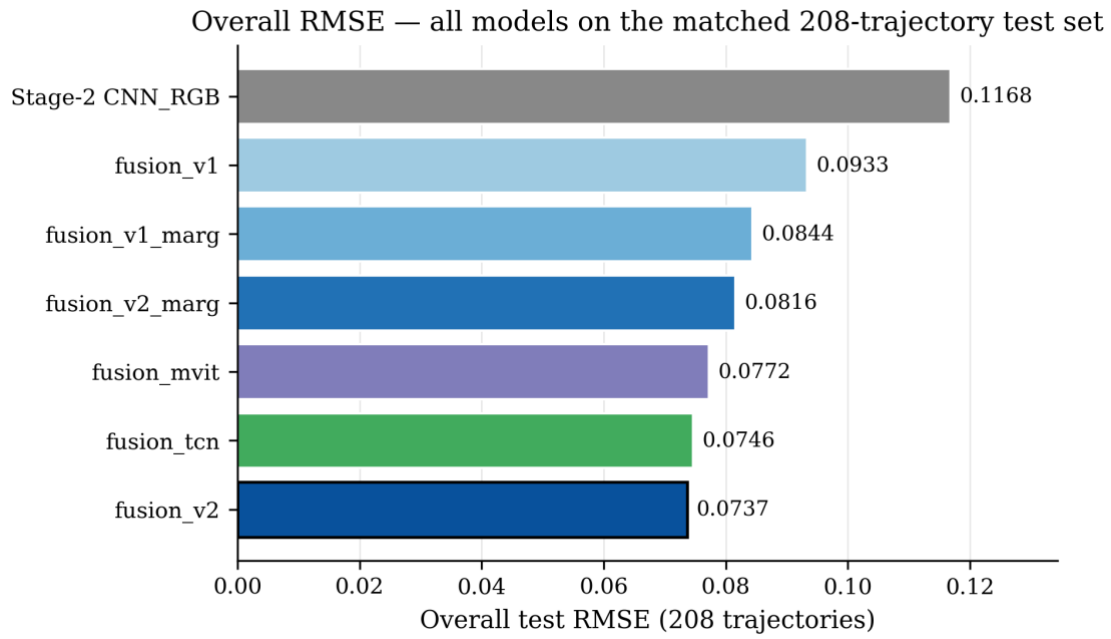


Figure 6: Overall RMSE of all models on test set

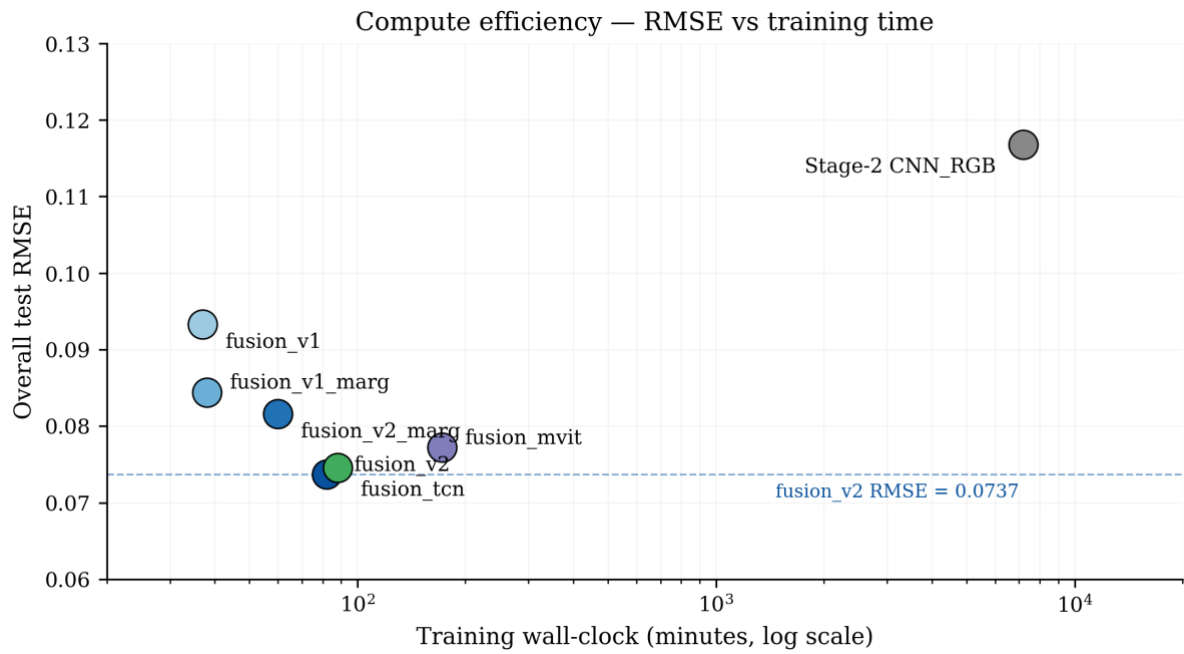


Figure 7: Compute efficiency

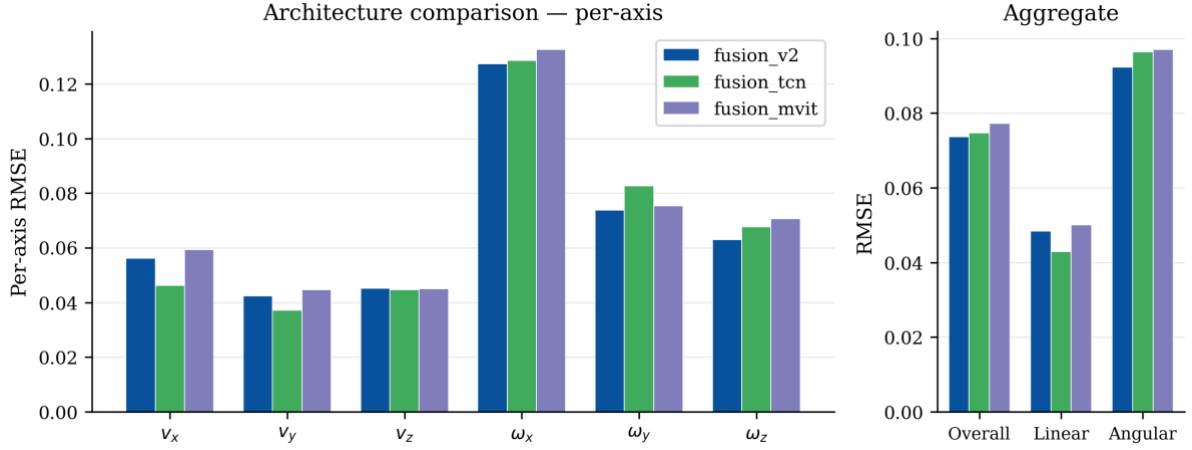


Figure 8: Architecture comparison per axis

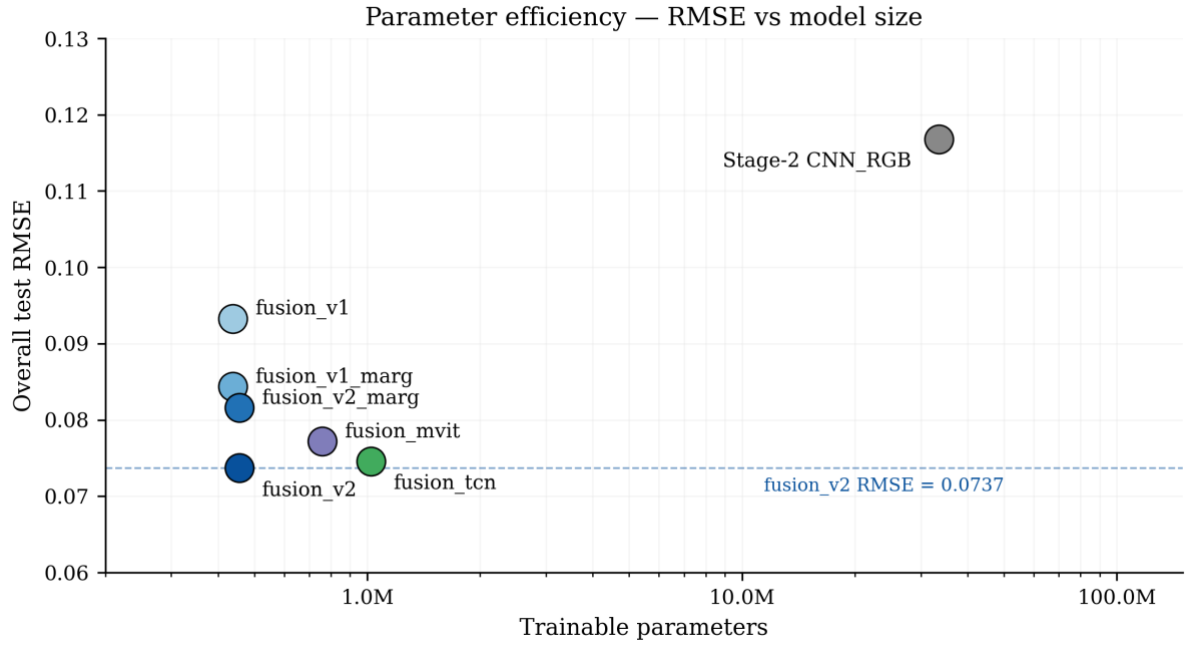


Figure 9: Parameter efficiency

5.5 Ablation Studies

Two controlled ablation studies were carried out, one a modality ablation (where the number and combination of active sensor arms were varied), and the other a temporal architecture ablation (comparing 3 different network architectures with the same input data).

5.5.1 Modality Ablation

The modality ablation holds the architecture constant (factorized transformer) and varies the input modalities. A summary of the four relevant models is given in Table 17.

Table 17: Summary of Four Relevant Models

Model	Kin	INS	VO	Overall	Linear (m/s)	Angular (rad/s)
fusion_v1	✓	IMU	—	0.0933	0.0637	0.1156
fusion_v1_marg	✓	MARG	—	0.0844	0.0586	0.1039
fusion_v2	✓	IMU	✓	0.0737	0.0483	0.0924
fusion_v2_marg	✓	MARG	✓	0.0816	0.0533	0.1023

Figure 10 shows the modality-ablation results.

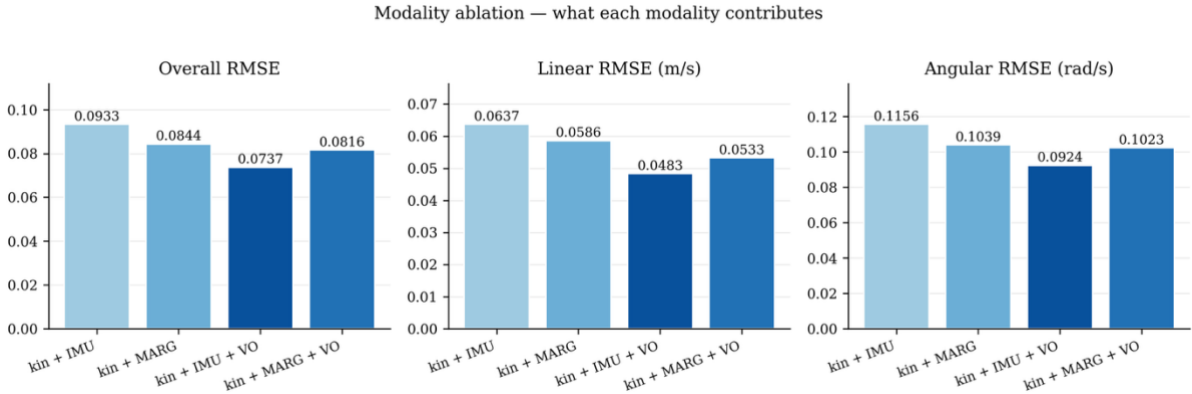


Figure 10: Modality ablation

VO contribution. By comparing the two versions of the fusion system, fusion_v1 and fusion_v2 (both with INS using only IMU), the visual odometry arm is isolated. Adding VO reduced overall RMSE from 0.0933 to 0.0737 (−21.0%), linear RMSE from 0.0637 to 0.0483 m/s (−24.2%), and angular RMSE from 0.1156 to 0.0924 rad/s (−20.1%). The greatest per-axis gain was on v_x (0.0786 m/s to 0.0562 m/s, −28.5%): DROID-SLAM's forward translation estimates are most reliable with their forward camera. The VO arm also improved ω_z (0.0806

to 0.0630 rad/s, -21.8%) through the SE(3) rotation component of the delta, which encodes frame-to-frame camera yaw.

Isolating the contribution of the MARG, requires two modality model. Comparing fusion_v1 with fusion_v1_marg is to isolate the magnetometer without VO present. The overall RMSE was reduced by 9.5% while the angular RMSE was reduced by 10.1% by MARG. The largest angular gain was on ω_x (0.1616 to 0.1391 rad/s, -13.9%), not ω_z as one might expect.

The most significant finding of the modality ablation is the "MARG paradox" between fusion_v2 and fusion_v2_marg. In fact, introducing the magnetometer into the three-modality system hurt the system: overall RMSE increased by 10.7% and performance degraded on every axis. This result is the most serious negative result of the study, being a total reversal of the two-modality results. This regression is explained by representational redundancy. In previous two modality tests, the MARG INS was the only yaw reference, but with the addition of the VO arm, this changed. The VO's Δq quaternion already provides a continuous stream of frame-to-frame yaw rotation. The introduction of MARG INS introduces a conflict between two yaw signals with very different scales, calibrations and noise characteristics: magnetometer declination offset and rotation accumulation from SLAM. The attention mechanism doesn't have a clear heuristic to decide which source is more reliable for any given clip, so the fusion process is just a compromise between the two. This effectively dilutes the high precision VO signal, and adds the inherent artefacts of the magnetometer. In conclusion, the results show that a sensor that is beneficial in a two-modality system can become a hindrance in a three-modality system when a more accurate pathway already measures the same physical quantity.

5.5.2 Temporal Architecture Ablation

The architecture ablation was performed all inputs, training configurations, and modality content fixed at the fusion_v2, specifically using kinematics, INS-IMU and Visual Odometry while varying only the temporal modelling network. Three architectures were compared in this study and they are:

- Factorised causal transformers in fusion_v2

- 1D temporal CNN in fusion_tcn
- Multiscale vision transformer with progressive temporal pooling in fusion_mvit

The comparative performance metrics are shown in the Table 18.

Table 18: Performance Metrics

Model	Overall RMSE	Linear RMSE (m/s)	Angular RMSE (rad/s)	Params	Train
fusion_v2 (transformer)	0.0737	0.0483	0.0924	455K	82 min
fusion_tcn (1D CNN)	0.0746	0.0429*	0.0964	1.02M	88 min
fusion_mvit (multiscale)	0.0772	0.0501	0.0970	759K	172 min

5.5.2.1 Transformer vs. TCN

The two architectures have different inductive biases, which are emphasized by a distinct split in the performance across axes. The fusion_tcn achieved the lowest linear RMSE in this study which is 0.0429 m/s, outperforming fusion_v2 across all three translation parameters. It improved longitudinal tracking (v_x) by 17.8%, lateral tracking (v_y), by 12.5%, and vertical tracking (v_z) by 1.3%. Conversely, the transformer-based fusion_v2 excelled across all rotational components, leading in roll (ω_x : 0.1272 vs. 0.1285 rad/s), pitch (ω_y : 0.0737 vs. 0.0826 rad/s), and yaw (ω_z : 0.0630 vs. 0.0676 rad/s) estimation.

This divergence was based on how each network structures its temporal operations. The 1D CNN adopts stride-2 downsampling to create a hierarchical temporal structure that is suitable for the highly regular gait of locomotion at 2 Hz. Since the 30 Hz kinematic and inertial streams are highly autocorrelated at 15 frames, the static, localized receptive fields of the convolutional layers are very efficient at isolating this periodic structure to perform linear velocity predictions.

Rotational dynamics, however, are dominated by non-periodic, transient events such as abrupt substrate shifts, varying footfall timings, or sudden losses of contact. A model is needed to focus on specific and informative frames across arbitrary time to track these dynamics. These signals are sparse and high frequency, and can be easily blurred by a downsampled convolutional window, but the self-attention mechanism in the factorized transformer captures them natively, regardless of their position in the sequence. This enables the transformer to keep its tracking edge across all the angular axes and at the same time be a smaller, faster model.

5.5.2.2 Transformers vs. MViT

The performance for the fusion_mvit architecture is inferior to fusion_v2 on all the tracking axes except one, for which it shows the slightest improvement (vertical linear velocity (v_z : 0.0450 vs. 0.0453 m/s). Moreover, its training time, 172 min, is more than double the training time of fusion_v2 (82 min). This poor performance is due to the multiscale design, which sequentially reduces the number of temporal tokens in each of three stages within the system ($T = 40 \rightarrow 20 \rightarrow 10$). Spatial-temporal downsampling of this kind offers a recipe for the true computational bottleneck of video understanding problems by performing a hierarchical compression of thousands of patch tokens, while causing crucial inefficiencies in low dimensional fusion problems from sensor readings.

In this setting, an input footprint of 120 tokens ($40 \text{ timesteps} \times 3 \text{ modalities}$) is reasonable without having to deal with the high-dimensional "token-count explosion". In fusion_mvit, by enforcing mean-pooling operations between successive and consecutive stages, fusion_mvit discards the fine-grained temporal information that can be natively yielded by the full-sequence attention layers in fusion_v2. Finally, all the parametric and computational overhead are provided by the multiscale pyramid without the receptive field benefit, it brings in high-token-count settings in vision. The empirical result is obvious: When there are few modalities and the windows which are fused are short, a hierarchical temporal reduction is not helpful.

5.5.3 Single-Modality Baselines

Single-modality baselines were tested on the same test set of 208 trajectories as a way of quantifying the standalone contribution of each preprocessing branch. The results of these are compared in with fusion_v2 Table 19.

Table 19: Single-modality Baselines vs. Fusion_v2

Modality	Linear RMSE (m/s)	Angular RMSE (rad/s)	Speed-mag RMSE (m/s)
Kin only ($v_{\text{body_legs}}$)	0.2464	-	0.2097
IMU only ($\text{gyro}_{\text{body}}$)	-	0.2944	-
VO only (DROID, stride=3)	1.0452	5.1825	1.7115
fusion_v2	0.0483	0.0924	-

Table 20 reports the per-axis breakdown for the kinematics and IMU baselines.

Table 20: Per-axis Breakdown for Kinematics and IMU

	v_x	v_y	v_z	ω_x	ω_y	ω_z
Kin only	0.2820	0.2316	0.2213	-	-	-
IMU only	-	-	-	0.4657	0.1730	0.1150
fusion_v2	0.0562	0.0425	0.0453	0.1272	0.0737	0.0630

Figure 11 compares the single-modality baselines against the full fusion model.

Single-modality baselines vs learned fusion (fusion_v2)

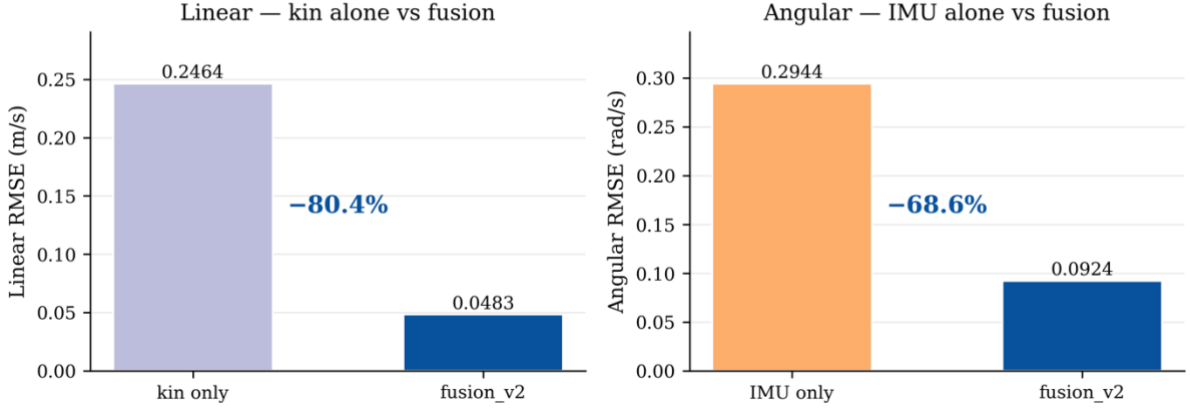


Figure 11: Single modality vs fusion

The kinematic-only baseline yielded an RMSE of 0.2464 m/s – about 5.1 times larger than the error of fusion_v2. The finite difference noise is amplified by the stance-foot constraint [68] at 30 Hz and a systematic error is introduced in the velocity estimate by leg compression under body roll oscillation. The errors seem to be fairly evenly distributed among the translational axes (0.22 to 0.28 m/s), suggesting that the kinematic baseline is better capturing the overall intensity of the locomotion than the precise direction of the motion.

The baseline that was based on IMU only achieved an angular RMSE of 0.2944 rad/s which was around 3.2 times higher than fusion_v2. The per-axis analysis shows that there is a significant difference between the performance of the yaw ($\omega_z = 0.1150$ rad/s) and pitch ($\omega_y = 0.1730$ rad/s) estimation, and the roll ($\omega_x = 0.4657$ rad/s) estimation. Such behaviour is probably related to a natural roll oscillation of the quadruped when executing the gait. The 2 Hz body roll pattern, together with the oscillation amplitude close to $\pm 5^\circ$, creates periodic oscillations of the angular velocity which are recorded quite accurately by the gyroscope. The ground-truth body-frame velocity labels, however, are temporally smoothed which results in averaging out parts of this oscillatory motion. As a result, the gyroscope measurements provide a more accurate picture of the actual physical motion than the supervision targets themselves and cause roll axis error measurements to be high.

The baseline of the visual odometry had the worst performance alone with 1.7115 m/s speed magnitude RMSE. This is an $\sim 1.95\times$ scale bias, which was probably a combination of stride-3 interpolation and imperfect stereo-scale recovery. Moreover, the original distribution of DROID-SLAM, which consists of TartanAir sequences with substantially larger inter-frame motions [75], than those produced by quadruped locomotion at 30 Hz, differs considerably from the Go2 forest sequences. In spite of these limitations, the visual odometry branch provides valuable motion information in terms of direction in the context of fusion_v2. The fusion network seems to make the use of the VO stream basically for providing the relative motion structure and consistency of the heading. This means the multimodal fusion is not only to further select the best modality, but it is also learning to fuse the complementary information from multiple modalities of independently noisy sensing streams.

5.6 Discussion

This section discusses the broader implications of the experimental results, including the following areas:

- structural limitations of end-to-end visual estimation,
- the architectural importance of the structured preprocessing
- the complex physical interactions between sensing modalities
- the remaining sources of estimation error within the proposed fusion framework

5.6.1 The Fundamental Limits of Visual Estimation

The initial visual experiments, spanning different backbone architecture and different visual input representations collectively found that forward-facing visual inputs cannot reliably recover all six body-frame velocity components. This is not due to model capacity limits but due to geometric observability constraints in this performance floor. At the scale of typical quadruped gait, a forward-facing camera will see strong longitudinal (v_x) optic expansion signals during forward locomotion, and strong yaw (ω_z) sweep signals during yaw rotations,

but only weak, potentially confusing signals of lateral translation, or chassis roll. These are deficiencies at the observability level and not a lack of expressiveness in the network. This means that the addition of more parameters to the model or the use of stereo depth maps created from images rather than the original RGB image will also not bring back information that was never there in the first place. The general consequence for mobile robotics is obvious: forward looking egocentric vision should not be used as a main source, but treated as an auxiliary signal to be optimized for the longitudinal velocity and yaw rate estimation.

5.6.2 The Value of Structured Preprocessing

The $73\times$ parameter reduction and $88\times$ training speedup of fusion_v2 over Stage-2, together with a 36.9% improvement in overall RMSE. This demonstrate the benefits of using structured preprocessing instead of the model trying to learn directly from the raw sensor data. The forward kinematics provides body velocity in physical units via a closed-form URDF computation. The Madgwick AHRS [73] filter integrates angular rates into a stable orientation estimate, and has been refined for well-established of signal processing. DROID-SLAM also aims to recover camera motion from frame-to-frame correspondences by learning a flow network to explicitly solve for camera motion. All of these structural transformations push raw sensor data into a representation, already semantically similar to the prediction task. The learning problem of the fusion network then can be expressed as temporal weighting and cross-modal arbitration, which is much smaller and better-conditioned than learning the complete sensor-to-velocity mapping from scratch.

5.6.3 Signal Redundancy

The modality ablation showed that it is not straightforward to enhance the performance of the pipeline by adding more sensors. The MARG paradox is an active phenomenon caused by representational redundancy between MARG and VO modalities. Adding the magnetometer only helps in the two modality system. But it hurts in the three modality system, where visual odometry already supplies yaw. This has direct implications for what could be done in the future when designing a sensor suite: not only must it be determined if a new modality will

provide useful information, but if it will provide complementary information to the current sensor suite.

5.6.4 Remaining Error Budget

The axis with the highest angular RMSE (0.1272 rad/s) of fusion_v2 is the roll axis, which is indicated by the rate ω_x . This lack of estimation is also heavily biased towards the tail of the distribution, with the P95 ω_x error of 0.1958 rad/s to the median of 0.0556 rad/s. This ratio of 3.5x indicates that errors are concentrated within minority of challenging frames rather than distributed evenly. The per trajectory data shows that 90% of the test trajectories have ω_x RMSE of 0.106 rad/s or lower, and only about 5% go past 0.25. This means the issue is a frame-level effect at specific moments, rather than just a few completely ruined trajectories. These tail frames are correlated with gait transition frames at foot contact and foot lift. At these moments the body's 2Hz roll oscillation peaks. The gyro captures the sharp spikes in roll rate perfectly. But the temporally smoothed ground-truth label averages them out. Because of this, the supervision target diverges most from the true motion at these precise frames.

The single-modality VO baseline confirms that the visual odometry arm cannot provide the system with the angular reference needed on this axis ($\omega_x = 5.59$ rad/s when the camera is approximately aligned with the body), leaving the IMU gyro as the primary structured source of roll. To address this instability, a velocity estimator based on contact events and conditions would have to be used. One that carries over the previous velocity estimate across a period of contact events (the transition window). Alternatively, short window IMU integration would have to be used during the contact events where gyroscope drift over a single gait cycle would be negligible.

On the opposite hand the velocity at 0.0562 m/s in forward direction (v_x) is the largest residual among the linear axes. This linear RMSE is the result of the scale uncertainty of DROID-SLAM on short-baseline quadruped sequences, from which it comes. The residual gap would most probably be closed even if the VO model were directly adapted to Go2 video by avoiding the need to change the core fusion architecture.

5.7 Real World Test Results

The simulation trained models that were tested on the real world datasets were fusion_v1, fusion_v2, fusion_tcn, and fusion_mvit. The results are presented in Table 21.

Table 21: Real-World Test Results.

Model	Overall RMSE	Linear RMSE (m/s)	Angular RMSE (rad/s)
fusion_v1	0.39	0.36	0.43
fusion_v2	0.37	0.37	0.36
fusion_tcn	0.58	0.53	0.63
fusion_mvit	0.39	0.41	0.38

The results on the real world dataset were significantly worse than the results from the simulation dataset. This might be explained by differences between the simulation and real world- including the sensor characteristics, commands given during simulation and data collection, environmental differences and inaccuracies in the hardware modelling. Domain randomization across the colour, geometry, lighting, sensor inputs and physical models may help to bridge the simulation to real world gap in the future. In addition, pretraining in simulation and fine-tuning with the real data may be an option in future work.

5.8 Summary

This chapter demonstrated the end-to-end process of visual estimation, from raw sensor learning to structured preprocessing and fusion, which was able to achieve a 36.9% decrease in overall RMSE with $73\times$ less parameters and $88\times$ faster learning. The visual odometry arm was the single most important factor contributing to that gain. And the MARG ablation showed that including the MARG in addition to the VO arm does not add additional orientation information that improves performance. The architecture comparison demonstrated that the 1D temporal CNN outperformed the factorised causal transformer and multiscale transformer in terms of linear velocity accuracy, and the factorised causal transformer is superior to the two other architectures in terms of angular accuracy, whereas the multiscale transformer is not superior. Roll rate on the ω_x axis continued to be the most significant residual limitation for all models evaluated. Finally, simulation-trained

models were evaluated on a real-world Unitree Go2, and performance degradation was noticed keeping significant sim-to-real gap.

6.0 Conclusion

This chapter summarizes the main findings of the study along with the research questions. The initial direction of this research explored end-to-end visual learning approaches with RGB images together with raw sensor data. However, it was found out that there were significant limitations when conducting the experiments on large scale dataset. This large scale experiments caused high computational cost, scalability issues and issues with angular motion estimation accuracy. This resulted in the shift to a structured pre-processing and fusion system, with physically meaningful modality representation created prior to the fusion process.

This thesis explored the 6-DoF body-frame velocity estimation of the Unitree Go2 quadruped through both visual and proprioceptive sensing in simulation. A total of 6 variants of the fusion models were tested along with the Stage-2 CNN RGB baseline on a 1,008-trajectory dataset obtained in Isaac Sim. Five research questions were considered in the research related to end-to-end visual architectures, visual input modalities, preprocessing based fusion, temporal fusion design, and sim-to-real transfer.

6.1 Question 1: CNN vs Transformers

The primary research question examined whether a 3D convolutional neural network or a multiscale video transformer provides a stronger foundation for end-to-end velocity estimation. The results from the pilot experiments showed that neither architecture was consistently dominant for all the modalities. Instead, performance was strongly depended on the visual representation and the architecture.

The model of MViT with RAFT-Stereo disparity showed the best pilot test RMSE value of 0.2260, followed by CNN RGB configuration with 0.2328. Comparing two architectures by the same modality, the MViT architecture was better with disparity input, while the CNN was better with RGB input. These results show that depth based geometric representations were more successful for transformer based architectures while local texture and motion representations from RGB imagery were more successful for convolutional based architectures.

But when the models were scaled to the entire data set, a second limitation was discovered. While the CNN-based architecture was trained at full scale in about five days and yielded a test RMSE of 0.1168, the MViT model was not computationally feasible at full scale as of yet due to significantly longer training times and large memory consumption. More importantly, neither architecture achieved sufficiently accurate six-axis body-frame velocity estimation. Based on these findings, the research pivoted towards preprocessing based multimodal fusion.

6.2 Question 2: RGB vs RAFT - Stereo Disparity

The second research question aimed to examine whether RGB imagery or RAFT-Stereo disparity is a better visual representation for velocity estimation. The experiments revealed that, again, this would depend on the architecture used.

RAFT-Stereo disparity performed better than RGB input for the MViT architecture. The disparity representation was able to explicitly provide geometric depth information that the transformer architecture could effectively fuse using global self-attention over the spatial field. In contrast, the CNN architecture achieved better performance using RGB imagery. The convolutional filters seemed to take advantage of the texture and motion gradients that were found within the forest environment, while the disparity input stripped away the colour and texture information that local convolutional feature extraction was using.

Within the final preprocessing-and-fusion pipeline, visual information was represented instead using DROID-SLAM SE(3) motion estimates. This provided a compact geometric representation extracted from stereo imagery while avoiding the computational cost of running dense disparity estimation during model training.

6.3 Question 3: Sim to Real Transfer.

The third research question investigated whether such a model trained completely with simulated data would be as accurate when applied to real-world data. To check on this, the simulation-trained models were tested on a real-world Unitree Go2 dataset (Section 5.7). The findings showed a significant difference in performance. The best model (fusion_v2) had an overall RMSE of 0.37 on real-world data compared with 0.0737 in simulation, approximately

five times less accurate than in simulation. There was also a comparable drop for the rest of the architectures: fusion_v1 (0.39), fusion_mvit (0.39), and fusion_tcn (0.58). Thus, the models were not able to retain the same accuracy when transferred to the real world. The performance decrease is attributed to discrepancies between the characteristics of the sensors, the hardware models, the locomotion commands, and the environment. These results suggest that although structured preprocessing and fusion is very successful in simulation, domain randomization alone is not sufficient to fully close the sim-to-real gap, and fine-tuning in the real world would likely be needed to regain accuracy.

6.4 Question 4: End to End vs Preprocessing and Fusion.

The fourth research question investigated whether end-to-end multimodal learning from raw image and sensor streams can be outperformed by structured preprocessing and learned fusion. The findings provided a clear answer to this question.

Even without any visual modality, fusion_v1 reduced overall RMSE by 20.1% compared to the Stage-2 CNN RGB baseline. After using the visual odometry arm in pipeline for fusion_v2, the overall RMSE value further decreased to 0.0737, which is a 36.9% improvement from the baseline from Stage-2. The gains were made with just about 73 times fewer parameters and in 82 minutes of training, compared with five days of training. The improvement was due to the use of physically meaningful levels of preprocessing in the estimation problem. The forward kinematics transformed the joint states to body motion estimates, the Madgwick filter extracted stable orientation features from IMU measurements, and DROID-SLAM determined geometric camera motion from stereo imagery. The fusion network therefore no longer needed to learn visual ego-motion, inertial integration, and kinematic reasoning directly from raw data. Instead, it learned to integrate structured modality representations over time. The experiments also demonstrated that the incorporation of extra sensing modalities is not always beneficial. The MARG experiments showed that magnetometer information improved estimation when only two modalities were present, but degraded performance once visual odometry was added. This shows that redundant orientation information from multiple independently-biased sources can negatively affect [76] learned fusion systems rather than improving the result.

6.5 Question 5 : Optimal Fusion Architecture

The final research question investigated which temporal fusion architecture produced the best overall performance for the preprocessing-based pipeline. The proposed FusionTransformer architecture was compared with two other architectures, temporal convolutional network and multiscale transformer, under the same experimental settings.

The fusion_v2 proposed architecture gave the best overall result, with an RMSE of 0.0737 and these were the lowest angular velocity errors of all models tested. The fusion_tcn architecture was the best performing linear velocity RMSE in the work and temporal convolution was very powerful to model periodic locomotion behavior. It still didn't perform as well as fusion_v2, however, in terms of angular estimation.

fusion_mvit architecture obtained competitive performance, however, no improvement in overall accuracy was obtained with fusion_mvit construction compared to fusion_v2 and the training time was significantly longer. The results indicate the factorized causal attention mechanism employed by fusion_v2 maintained the temporal and modal relationships better than fusion_mvit's multiscale pooling. Fusion_v2 performed best in terms of accuracy of estimation, complexity, number of parameters, and deployment readiness, making it the recommended fusion architecture.

6.6 Engineering Contribution.

In addition to the modeling contributions, this thesis produced several engineering contributions. A large-scale quadruped simulation dataset was created in Isaac Sim with stratified terrain coverage of forest, flat and uneven terrain, comprising 1,008 trajectories. The data consisted of synchronized stereo images, IMU and magnetometer data, joint states and six-axis body-frame velocity labels.

6.7 Limitation and Future Work

Roll rate estimation was the remaining constraint in the proposed fusion system. fusion_v2 significantly outperformed the individual modality baseline results in terms of angular estimation,

however, the roll axis still dominated the estimation results. Addressing this axis more precisely represents a clear direction for future work.

A second issue with the visual odometry arm is the scale uncertainty, due to the difference between the DROID-SLAM training distribution and slower inter-frame motion patterns of the quadruped platform. The fusion architecture was able to mitigate much of this error during training, but adapting the visual odometry pipeline to Go2-specific data should be able to further reduce this error with regards to translation.

While much domain randomization was done during the data collection process, sim-to-real transfer is a key element to investigate in the future. Thus, it is necessary to test the proposed fusion framework more on the real Unitree Go2 on physical outdoor environments in the future. Further research can focus on contact-aware kinematic estimators, light-weight real-time visual odometry pipelines, adaptive modality weighting and self-supervised online adaptation methods for long-term deployment.

6.8 Conclusion

The results of this thesis show that structured preprocessing coupled with lightweight temporal fusion results in a significant performance improvement compared to end-to-end multimodal learning from raw RGB image and raw sensor stream in this experimental setup for the task of quadruped state estimation.

The study showed that CNNs and transformers are not universally ordered, but instead interact strongly with the chosen visual representation. Although different architectures achieved greater performance in RGB and stereo disparity, respectively, fusion using preprocessing consistently achieved higher performance than direct end-to-end learning with both in terms of accuracy and efficiency. The results showed that the proposed factorized causal FusionTransformer outperformed all the other fusion models tested in this work under different aspects of comparison including overall accuracy, angular velocity accuracy, the parameter efficiency, and the deployment suitability.

The pre-processing pipeline, multimodal fusion framework, trained fusion architectures and large scale simulation dataset created for this work lay the ground work for future work for robust proprioceptive and visual state estimation for legged robotic systems.

Bibliography

- [1] M. Yuan, “Design and Development of an Intelligent Quadruped Rescue Robot for Disaster Response,” in *2025 5th International Conference on Sensors and Information Technology*, 2025, pp. 740–743. doi: 10.1109/ICSI64877.2025.11009828.
- [2] Z. Betta, A. Gaudino, A. Benini, C. T. Recchiuto, and A. Sgorbissa, “Perceptions and Opinions of Rescuers about a Quadruped Robot in an Earthquake Scenario,” in *2024 33rd IEEE International Conference on Robot and Human Interactive Communication (ROMAN)*, 2024, pp. 1092–1099. doi: 10.1109/RO-MAN60168.2024.10731200.
- [3] S. Solmaz *et al.*, “Robust Robotic Search and Rescue in Harsh Environments: An Example and Open Challenges,” in *2024 IEEE International Symposium on Robotic and Sensors Environments (ROSE)*, 2024, pp. 1–8. doi: 10.1109/ROSE62198.2024.10591144.
- [4] A. Khasawneh, H. Rogers, J. Bertrand, K. C. Madathil, and A. Gramopadhye, “Human adaptation to latency in teleoperated multi-robot human-agent search and rescue teams,” *Autom. Constr.*, vol. 99, pp. 265–277, 2019, doi: 10.1016/j.autcon.2018.12.012.
- [5] Z. Betta, S. Paneri, A. Gaudino, A. Benini, C. T. Recchiuto, and A. Sgorbissa, “Multi-floor danger and responsiveness assessment with autonomous legged robots in catastrophic scenarios,” in *2023 32nd IEEE International Conference on Robot and Human Interactive Communication (RO-MAN)*, 2023, pp. 1063–1070. doi: 10.1109/RO-MAN57019.2023.10309412.
- [6] R. Parker, K. Bayne, and P. Clinton, “Robotics in forestry,” *New Zealand Journal of Forestry*, vol. 60, pp. 8–14, Feb. 2016.
- [7] Y. Nisticò, J. C. V. Soares, L. Amatucci, G. Fink, and C. Semini, “MUSE: A real-time multi-sensor state estimator for quadruped robots,” *IEEE Robot. Autom. Lett.*, vol. 10, no. 5, pp. 4620–4627, May 2025, doi: 10.1109/LRA.2025.3553047.

- [8] T. Zhang, K. Wu, J. Song, S. Huang, and G. Dissanayake, “Convergence and consistency analysis for a 3-D invariant-EKF SLAM,” *IEEE Robot. Autom. Lett.*, vol. 2, no. 2, pp. 733–740, 2017, doi: 10.1109/LRA.2017.2651376.
- [9] M. Camurri, M. Ramezani, S. Nobili, and M. Fallon, “Pronto: A multi-sensor state estimator for legged robots in real-world scenarios,” *Front. Robot. AI*, vol. 7, p. 68, 2020, doi: 10.3389/frobt.2020.00068.
- [10] A. Vecchione and T. Boaventura, “Performance Comparison of Common Low-level State Estimators for Legged Robots,” in *2025 Brazilian Conference on Robotics (CROS)*, 2025, pp. 1–5. doi: 10.1109/CROS66186.2025.11066151.
- [11] S. Teng, M. W. Mueller, and K. Sreenath, “Legged Robot State Estimation in Slippery Environments Using Invariant Extended Kalman Filter with Velocity Update,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 3104–3110. doi: 10.1109/ICRA48506.2021.9561313.
- [12] S. Ha, J. Lee, M. van de Panne, Z. Xie, W. Yu, and M. Khadiv, “Learning-based legged locomotion: State of the art and future perspectives,” *Int. J. Rob. Res.*, vol. 44, no. 8, pp. 1396–1427, 2025, doi: 10.1177/02783649241312698.
- [13] T. Anne, J. Wilkinson, and Z. Li, “Meta-Learning for Fast Adaptive Locomotion with Uncertainties in Environments and Robot Dynamics,” in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 4568–4575. doi: 10.1109/IROS51168.2021.9635840.
- [14] R. Buchanan, M. Camurri, F. Dellaert, and M. Fallon, “Learning inertial odometry for dynamic legged robot state estimation,” in *Conference on robot learning*, 2022, pp. 1575–1584. doi: 10.48550/arXiv.2111.00789.
- [15] X. Liu *et al.*, “Learning-Assisted Multi-IMU Proprioceptive State Estimation for Quadruped Robots,” *Information*, vol. 16, no. 6, p. 479, 2025, doi: 10.3390/info16060479.

- [16] L. Abdrazakov and D. Yudin, “Neural Network Adaptation of the Kalman Filter for Odometry Fusion,” 2021, pp. 44–54. doi: 10.1007/978-3-030-87178-9_5.
- [17] N. Sanghai and N. B. Brown, “Advances in Transformers for Robotic Applications: A Review,” 2024. doi: 10.48550/arXiv.2412.10599.
- [18] J. Tremblay *et al.*, “Training deep networks with synthetic data: Bridging the reality gap by domain randomization,” in *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*, 2018, pp. 969–977. doi: 10.48550/arXiv.1804.06516.
- [19] TorchVision maintainers and contributors, “resnet18 - Torchvision main documentation.” [Online]. Available: <https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html>
- [20] Y. Yu, A. Honglei, H. Yezhong, M. Hongxu, and L. Lin, “Moving Horizon Body State Estimation for Quadruped Robot with Multi-Sensor Information Fusion,” in *2020 7th International Conference on Information Science and Control Engineering (ICISCE)*, 2020, pp. 861–868. doi: 10.1109/ICISCE50968.2020.00179.
- [21] S. Fahmi, G. Fink, and C. Semini, “On State Estimation for Legged Locomotion over Soft Terrain,” *IEEE Sens. Lett.*, vol. 5, no. 1, 2021, doi: 10.1109/LSENS.2021.3049954.
- [22] T. Zhang, W. Cao, C. Liu, T. Zhang, J. Li, and S. E. Li, “Robust State Estimation for Legged Robots With Dual Beta Kalman Filter,” *IEEE Robot. Autom. Lett.*, vol. 10, no. 8, pp. 7955–7962, 2025, doi: 10.1109/LRA.2025.3579619.
- [23] M. Bloesch, C. Gehring, P. Fankhauser, M. Hutter, M. A. Hoepflinger, and R. Siegwart, “State estimation for legged robots on unstable and slippery terrain,” in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2013, pp. 6058–6064. doi: 10.1109/IROS.2013.6697236.

- [24] C. Gao, Y. Xie, L. Kong, X. Chen, A. Xie, and D. Zhang, "SVM-Based State Estimation of Biped Robot," in *2021 7th International Conference on Mechatronics and Robotics Engineering (ICMRE)*, 2021, pp. 41–47. doi: 10.1109/ICMRE51691.2021.9384850.
- [25] I. A. Kuncara, A. Widyotriatmo, A. Hasan, and Y. Y. Nazaruddin, "Enhancing accuracy in field mobile robot state estimation with GNSS and encoders," *Measurement*, vol. 235, p. 114903, 2024, doi: 10.1016/j.measurement.2024.114903.
- [26] T. Cvitanic, S. Melkote, and S. Balakirsky, "Improved state estimation of a robot end-effector using laser tracker and inertial sensor fusion," *CIRP J. Manuf. Sci. Technol.*, vol. 38, pp. 51–61, 2022, doi: 10.1016/j.cirpj.2022.03.011.
- [27] D. Youm, H. Oh, S. Choi, H. Kim, and J. Hwangbo, "Legged robot state estimation with invariant extended kalman filter using neural measurement network," *arXiv preprint arXiv:2402.00366*, 2024, doi: 10.48550/arXiv.2402.00366.
- [28] A. Albadin, C. Albitar, and M. Alsaba, "Estimation of the legs' state of a mobile robot based on Long Short-Term Memory network," *Eng. Appl. Artif. Intell.*, vol. 139, p. 109539, 2025, doi: 10.1016/j.engappai.2024.109539.
- [29] Z. Shi, "Incorporating Transformer and LSTM to Kalman Filter with EM algorithm for state estimation," *arXiv preprint arXiv:2105.00250*, 2021, doi: 10.48550/arXiv.2105.00250.
- [30] J. P. do Monte Lima, H. Uchiyama, and R. Taniguchi, "End-to-end learning framework for imu-based 6-dof odometry," *Sensors*, vol. 19, no. 17, p. 3777, 2019, doi: 10.3390/s19173777.
- [31] K. Yazdipaz, N. Kohli, S. Ali Golestaneh, and M. Shahbazi, "Robust and Efficient Phase Estimation in Legged Robots via Signal Imaging and Deep Neural Networks," *IEEE Access*, vol. 13, pp. 49018–49029, 2025, doi: 10.1109/ACCESS.2025.3549165.

- [32] A. Yang, Y. Chen, H. Jin, L. Chen, R. Chen, and Y. Xu, “CNN based Relocalization Method for Indoor Moving Robot with Planar State Constraint,” in *Chinese Control Conference, CCC*, IEEE Computer Society, Jul. 2021, pp. 4296–4301. doi: 10.23919/CCC52363.2021.9550606.
- [33] D. Sójka, M. R. Nowicki, and P. Skrzypczyński, “Learning an efficient terrain representation for haptic localization of a legged robot,” *arXiv preprint arXiv:2209.15135*, 2022, doi: 10.48550/arXiv.2209.15135.
- [34] A. Schperberg, Y. Tanaka, S. Mowlavi, F. Xu, B. Balaji, and D. Hong, “OptiState: State Estimation of Legged Robots using Gated Networks with Transformer-based Vision and Kalman Filtering,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 6314–6320. doi: 10.1109/ICRA57147.2024.10610160.
- [35] P. Dahal, S. Mentasti, L. Paparusso, S. Arrigoni, and F. Braghin, “RobustStateNet: Robust ego vehicle state estimation for Autonomous Driving,” *Rob. Auton. Syst.*, vol. 172, p. 104585, 2024, doi: 10.1016/j.robot.2023.104585.
- [36] Y. Gu, Z. Liu, and T. Zou, “Enhancing Leg Odometry in Legged Robots with Learned Contact Bias: An LSTM Recurrent Neural Network Approach,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 6832–6839. doi: 10.1109/IROS58592.2024.10802731.
- [37] Q. Zhao *et al.*, “A Sensor Fusion Algorithm: Improving State Estimation Accuracy for a Quadruped Robot Dog,” in *2022 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, 2022, pp. 525–530. doi: 10.1109/ROBIO55434.2022.10011894.
- [38] S. Song, B. Yu, M. Oh, and H. Myung, “BIG-STEP: Better-Initialized State Estimator for Legged Robots with Fast and Robust Ground Segmentation,” in *2023 23rd International Conference on Control, Automation and Systems (ICCAS)*, 2023, pp. 184–188. doi: 10.23919/ICCAS59377.2023.10316815.

- [39] J. Kang, Y. Wang, and X. Xiong, “Fast Decentralized State Estimation for Legged Robot Locomotion via EKF and MHE,” *IEEE Robot. Autom. Lett.*, vol. PP, pp. 1–8, Aug. 2024, doi: 10.1109/LRA.2024.3483043.
- [40] V. Agrawal, S. Bertrand, R. Griffin, and F. Dellaert, “Proprioceptive State Estimation of Legged Robots with Kinematic Chain Modeling,” Aug. 2022, pp. 178–185. doi: 10.1109/Humanoids53995.2022.10000099.
- [41] Y. Nisticò, H. Kim, J. C. V. Soares, G. Fink, H.-W. Park, and C. Semini, “Multi-Sensor Fusion for Quadruped Robot State Estimation Using Invariant Filtering and Smoothing,” *IEEE Robot. Autom. Lett.*, 2025, doi: 10.1109/LRA.2025.3564711.
- [42] NVIDIA, “Isaac Sim: Quadruped ROS tutorial,” 2022. Accessed: Jun. 05, 2026. [Online]. Available: <https://docs.isaacsim.omniverse.nvidia.com/latest/index.html>
- [43] CLeARoboticsLab, “Unitree Go2 Isaac ROS2 integration,” 2023. Accessed: Jun. 05, 2026. [Online]. Available: https://github.com/CLeARoboticsLab/go2_isaac_ros2
- [44] M. Körber, J. Lange, S. Rediske, S. Steinmann, and R. Glück, “Comparing popular simulation environments in the scope of robotics and reinforcement learning,” *arXiv preprint arXiv:2103.04616*, 2021, doi: 10.48550/arXiv.2103.04616.
- [45] G. Fink and C. Semini, “The DLS quadruped proprioceptive sensor dataset,” in *Proceedings of the 2020 International Conference on Climbing and Walking Robots (CLAWAR), Moscow, Russia*, 2020, pp. 24–26.
- [46] J. Yin, A. Li, T. Li, W. Yu, and D. Zou, “M2dgr: A multi-sensor and multi-scenario slam dataset for ground robots,” *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 2266–2273, 2022, doi: 10.1109/LRA.2021.3138527.
- [47] M. Patel *et al.*, “TartanGround: A Large-Scale Dataset for Ground Robot Perception and Navigation,” *arXiv preprint arXiv:2505.10696*, 2025, doi: 10.48550/arXiv.2505.10696.

- [48] K. M. Judd and J. D. Gammell, “The Oxford multimotion dataset: Multiple SE (3) motions with ground truth,” *IEEE Robot. Autom. Lett.*, vol. 4, no. 2, pp. 800–807, 2019, doi: 10.1109/LRA.2019.2892656.
- [49] H. Wei *et al.*, “Fusionportablev2: A unified multi-sensor dataset for generalized slam across diverse platforms and scalable environments,” *Int. J. Rob. Res.*, vol. 44, no. 7, pp. 1093–1116, 2025, doi: 10.1177/02783649241303525.
- [50] T. Pire, M. Mujica, J. Civera, and E. Kofman, “The Rosario dataset: Multisensor data for localization and mapping in agricultural environments,” *Int. J. Rob. Res.*, vol. 38, no. 6, pp. 633–641, 2019, doi: 10.1177/0278364919841437.
- [51] D. Wisth, M. Camurri, and M. Fallon, “Vilens: Visual, inertial, lidar, and leg odometry for all-terrain legged robots,” *IEEE Transactions on Robotics*, vol. 39, no. 1, pp. 309–326, 2022, doi: 10.1109/TRO.2022.3193788.
- [52] Torchvision Contributors, “Video ResNet — Torchvision,” 2025. Accessed: Jun. 05, 2026. [Online]. Available: https://docs.pytorch.org/vision/main/models/video_resnet.html
- [53] Z. Ren, X. Xiao, and H. Nie, “Empowering Efficient Spatio-Temporal Learning with a 3D CNN for Pose-Based Action Recognition,” *Sensors (Basel)*, vol. 24, no. 23, p. 7682, 2024.
- [54] Torchvision Contributors, “Video MViT — Torchvision,” 2025. [Online]. Available: https://docs.pytorch.org/vision/main/models/video_mvit.html
- [55] L. Sun, G. Ding, Y. Qiu, Y. Yoshiyasu, and F. Kanehiro, “TransFusionOdom: interpretable transformer-based LiDAR-inertial fusion odometry estimation,” *arXiv preprint arXiv:2304.07728*, 2023, doi: 10.48550/arXiv.2304.07728.

- [56] W. Yan, D. Hafner, S. James, and P. Abbeel, “Temporally consistent transformers for video generation,” in *International Conference on Machine Learning*, 2023, pp. 39062–39098. doi: 10.48550/arXiv.2210.02396.
- [57] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems*, Neural Information Processing Systems Foundation, 2021, pp. 16558–16569. doi: 10.48550/arXiv.2108.10869.
- [58] Z. Zhou *et al.*, “Towards building AI-CPS with NVIDIA Isaac Sim: An industrial benchmark and case study for robotics manipulation,” in *Proceedings of the 46th international conference on software engineering: software engineering in practice*, 2024, pp. 263–274. doi: 10.1145/3639477.3639740.
- [59] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World,” in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30. doi: 10.1109/IROS.2017.8202133.
- [60] H. Fan *et al.*, “Multiscale Vision Transformers,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 6804–6815. doi: 10.1109/iccv48922.2021.00675.
- [61] D. Tran, H. Wang, L. Torresani, J. Ray, Y. LeCun, and M. Paluri, “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 6450–6459. doi: 10.1109/CVPR.2018.00675.
- [62] W. Kay *et al.*, “The Kinetics Human Action Video Dataset,” *arXiv preprint*, May 2017, doi: 10.48550/arXiv.1705.06950.

- [63] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.
- [64] Y. Li *et al.*, “MViTv2: Improved Multiscale Vision Transformers for Classification and Detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 4794–4804. doi: 10.1109/CVPR52688.2022.00476.
- [65] L. Lipson, Z. Teed, and J. Deng, “RAFT-Stereo: Multilevel Recurrent Field Transforms for Stereo Matching,” in *International Conference on 3D Vision (3DV)*, 2021, pp. 218–227. doi: 10.1109/3DV53792.2021.00032.
- [66] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *European Conference on Computer Vision (ECCV)*, 2020, pp. 402–419. doi: 10.1007/978-3-030-58536-5_24.
- [67] J. Solà, J. Deray, and D. Atchuthan, “A Micro Lie Theory for State Estimation in Robotics,” *arXiv preprint arXiv:1812.01537*, 2018, doi: 10.48550/arXiv.1812.01537.
- [68] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *International Conference on Learning Representations (ICLR)*, 2019. doi: 10.48550/arXiv.1711.05101.
- [69] G. Bertasius, H. Wang, and L. Torresani, “Is Space-Time Attention All You Need for Video Understanding?,” in *International Conference on Machine Learning (ICML)*, 2021, pp. 813–824. doi: 10.48550/arXiv.2102.05095.
- [70] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2017, pp. 5998–6008. doi: 10.48550/arXiv.1706.03762.

- [71] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv preprint arXiv:1607.06450*, 2016, doi: 10.48550/arXiv.1607.06450.
- [72] A. El-Nouby *et al.*, “XCiT: Cross-Covariance Image Transformers,” *CoRR*, vol. abs/2106.09681, 2021, doi: 10.48550/arXiv.2106.09681.
- [73] S. O. H. Madgwick, A. J. L. Harrison, and R. Vaidyanathan, “Estimation of IMU and MARG orientation using a gradient descent algorithm,” in *2011 IEEE International Conference on Rehabilitation Robotics*, 2011, pp. 1–7. doi: 10.1109/ICORR.2011.5975346.
- [74] D. Wisth, M. Camurri, and M. Fallon, “Robust Legged Robot State Estimation Using Factor Graph Optimization,” *IEEE Robot. Autom. Lett.*, vol. 4, no. 4, pp. 4507–4514, 2019, doi: 10.1109/LRA.2019.2933768.
- [75] W. Wang *et al.*, “TartanAir: A Dataset to Push the Limits of Visual SLAM,” in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 4909–4916. doi: 10.1109/IROS45743.2020.9341801.
- [76] T. Baltrusaitis, C. Ahuja, and L.-P. Morency, “Multimodal Machine Learning: A Survey and Taxonomy,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 2, pp. 423–443, 2019, doi: 10.1109/TPAMI.2018.2798607.